

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Введение в нереляционные базы данных»
Тема: Сервис по вывозу мусора (Приложение одновременно для
курьеров и клиентов)

Студенты гр. 3388

Потоцкий С.С.

Снигирев А.А.

Титкова С.Д.

Преподаватель

Заславский М. М.

Санкт-Петербург

2026

ЗАДАНИЕ

Студенты:

Потоцкий С.С.

Снигирев А.А.

Титкова С.Д.

Группа 3388

Тема проекта: Разработка сервиса по вывозу мусора.

Исходные данные:

Необходимо реализовать мобильное приложение для сервиса вывоза мусора (единое для клиентов и курьеров) на базе ArangoDB.

Содержание пояснительной записки:

«Содержание»

«Введение»

«Качественные требования к решению»

«Сценарий использования»

«Модель данных»

«Разработка приложения»

«Вывод»

«Приложение»

Предполагаемый объем пояснительной записки:

Не менее 20 страниц.

Дата выдачи задания:

Дата сдачи реферата:

Дата защиты реферата:

Студенты гр. 3388

Потоцкий С.С.

Снигирев А.А.

Титкова С.Д.

Преподаватель

Заславский М. М.

АННОТАЦИЯ

В рамках данного курса был разработан прототип информационной системы «ЭкоСервис», предназначенной для автоматизации процесса управления вывозом бытовых и строительных отходов. Основной целью работы стало исследование возможностей мультимодельной СУБД ArangoDB [1] при реализации сложных связей между участниками системы: заказчиками, курьерами и администраторами.

Выбор данной СУБД обусловлен необходимостью эффективного хранения как структурированных данных о заказах, так и графовых связей для отслеживания цепочек исполнения заявок. В ходе разработки оценивались такие аспекты, как гибкость схемы данных и производительность графовых запросов (AQL) при фильтрации и поиске заказов.

SUMMARY

Within the framework of this course, a prototype of the "EcoService" information system was developed, designed to automate the management of domestic and construction waste removal. The primary objective of the project was to explore the capabilities of the ArangoDB [1] multi-model database in implementing complex relationships between system participants: customers, couriers, and administrators.

The choice of this DBMS was driven by the necessity for efficient storage of both structured order data (document model) and graph relationships for tracking the execution chains of service requests. During the development process, aspects such as data schema flexibility and the performance of graph queries (AQL) for filtering and searching orders were evaluated.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	6
1. КАЧЕСТВЕННЫЕ ТРЕБОВАНИЯ К РЕШЕНИЮ	6
2. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ	7
2.1. Макет UI	7
2.2. Сценарии использования для задач	9
3. МОДЕЛЬ ДАННЫХ	27
3.1 Нереляционная модель данных.....	27
3.2. Реляционная модель данных	38
3.3 Сравнение моделей	44
3.4. Сравнительный анализ теоретических и практических объёмов хранения данных.....	46
4. РАЗРАБОТКА ПРИЛОЖЕНИЯ.....	49
4.1. Краткое описание.....	49
4.2. Используемые технологии	51
4.3. Экраны приложения.....	51
5. ВЫВОД.....	56
5.1. Результаты.....	56
5.2. Недостатки и пути для улучшения полученного решения	57
5.3. Будущее развитие решения	57
6. ПРИЛОЖЕНИЕ.....	58
ИСПОЛЬЗУЕМАЯ ЛИТЕРАТУРА	58
ПРИЛОЖЕНИЕ А.....	59

ВВЕДЕНИЕ

В условиях современного города объемы бытовых и строительных отходов постоянно растут, что требует более гибких и быстрых инструментов управления их утилизацией. Существующие методы взаимодействия между заказчиками и частными перевозчиками часто неавтоматизированы, что приводит к задержкам в обработке заявок и отсутствию прозрачности в логистике. Разработка цифровой платформы для прямого взаимодействия участников процесса позволяет оптимизировать распределение заказов и улучшить экологическую ситуацию.

Целью работы является создание высокопроизводительного веб-приложения «ЭкоСервис» для автоматизации процесса приема, распределения и контроля исполнения заявок на вывоз мусора. Система должна обеспечивать три уровня доступа (заказчик, курьер, администратор) и поддерживать работу с динамически меняющимися данными о заказах и адресах.

1. КАЧЕСТВЕННЫЕ ТРЕБОВАНИЯ К РЕШЕНИЮ

Было решено разработать облачное веб-приложение на базе современного стека технологий: React для фронтенд-части и FastAPI для серверной логики. В качестве хранилища данных выбрана мультимодельная СУБД ArangoDB, которая позволяет одновременно использовать документный подход (для профилей пользователей) и графовый подход (для моделирования связей между исполнителями, заказами и локациями).

Требуется разработать отказоустойчивое приложение, соответствующее следующим критериям:

- Использование графовых связей для реализации сложной логики назначения заказов и отслеживания их истории.
- Высокая скорость выполнения поисковых запросов и фильтрации данных с помощью языка AQL.
- Обеспечение интуитивно понятного UI/UX для мобильных и десктопных устройств.

2. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

2.1. Макет UI

В ходе разработки системы автоматизации вывоза отходов был спроектирован и реализован многопользовательский веб-интерфейс. Архитектура интерфейса разделена на общие модули (авторизация и регистрация) и три специализированных личных кабинета в соответствии с ролевой моделью системы: заказчик, курьер и администратор. Макеты всех страниц вынесены в Приложение А.

Вход в систему осуществляется через унифицированную форму, содержащую механизмы перенаправления на регистрацию и восстановление доступа (*Приложение А, рисунок А.1, А.6*). При регистрации пользователь проходит этап выбора роли (*Приложение А, рисунок А.2*), после чего заполняет специализированную форму:

- *Регистрация заказчика:* содержит базовые персональные данные и адрес проживания (*Приложение А, рисунок А.3*).
- *Регистрация курьера:* расширена полями выбора транспортного средства (пеший, легковой автомобиль, грузовой автомобиль) и формой загрузки верификационных документов (паспорта) (*Приложение А, рисунок А.4*).

- *Регистрация администратора:* требует ввода стандартных учетных данных и индивидуального пароля доступа (*Приложение А, рисунок А.5*).

Процедура восстановления доступа построена по двухэтапному принципу с отправкой проверочного кода на электронную почту и последующей установкой нового пароля (*Приложение А, рисунок А.25*).

Личный кабинет заказчика ориентирован на быстрое создание заявок и мониторинг расходов. На главной странице отображаются текущий баланс, оперативная аналитика (заказы в пути, выполненные, траты за месяц) и список активных заявок (*Приложение А, рисунок А.15*). Процесс создания заказа снабжен интерактивным бегунком для выбора объема отходов и калькулятором стоимости в реальном времени (*Приложение А, рисунок А.16*). Для детального анализа предусмотрены экраны просмотра параметров конкретного заказа (*Приложение А, рисунок А.17*), журнал всех созданных заявок с фильтрацией (*Приложение А, рисунок А.18*) и форма онлайн-пополнения счета (*Приложение А, рисунок А.19*). Редактирование личных данных и адреса доступно в профиле (*Приложение А, рисунок А.20, А.14*).

Рабочее пространство курьера спроектировано для работы «в поле». Главная страница предоставляет доступ к балансу и ленте доступных заказов с фильтрацией по категориям отходов (*Приложение А, рисунок А.7*). Экран взятия заказа отображает исчерпывающие данные, включая комментарий и стоимость (*Приложение А, рисунок А.8*).

Процесс выполнения заказа защищен сквозным контролем: для закрытия заявки курьер обязан загрузить фото подтверждения выполненной работы (*Приложение А, рисунок А.10*). Финансовый блок курьера включает расширенную страницу статистики с карточками доходов, графиками динамики (в сравнении с прошлым месяцем), историей

транзакций (*Приложение А, рисунок А.11*) и интерфейсом вывода средств на банковскую карту (*Приложение А, рисунок А.12*). Управление личными данными и историей выполненных заказов осуществляется через соответствующие экраны (*Приложение А, рисунок А.9, А.13*).

Панель администратора выполняет функции верхнеуровневого контроля и построения сквозной аналитики. На основном дашборде выводятся ключевые метрики за текущие сутки (количество заказов, курьеры на линии, общий объем вывезенного мусора в m^3) и живая лента системных событий (*Приложение А, рисунок А.21*).

Для операционной работы реализованы два диспетчерских экрана:

- *Управление заказами*: таблица с фильтрацией по статусам и типам отходов (*Приложение А, рисунок А.22*).
- *Управление пользователями*: инструмент модерации аккаунтов с возможностью поиска по ФИО, телефону и фильтрацией по ролям (*Приложение А, рисунок А.23*).

Блок «Аналитика» предоставляет конструктор отчетов, позволяющий гибко настраивать оси координат (Х — типы мусора, временные периоды, курьеры; Y — объемы, количество, финансовые показатели) для построения динамических диаграмм (*Приложение А, рисунок А.24*).

2.2. Сценарии использования для задач

В данном приложении реализовано три роли: заказчик, курьер и администратор.

Разберем общие сценарии использования:

1. Регистрация нового пользователя и выбор роли

Действующее лицо: Незарегистрированный пользователь

Страницы: Главная страница → Выбор роли → Регистрация заказчика /
Регистрация курьера / Регистрация администратора

- **Предусловие:** Пользователь не авторизован в системе, находится на главной странице сайта.
- **Основной сценарий:**
 - i. Пользователь нажимает кнопку «Создать аккаунт» на главной странице.
 - ii. Система открывает страницу с выбором роли (заказчик или курьер).
 - iii. В зависимости от выбора роли система открывает форму регистрации.
 - iv. Пользователь вводит имя, email, номер телефона и придумывает пароль; если регистрируется как курьер — прикладывает фото документа и выбирает тип транспорта.
 - v. Пользователь нажимает кнопку «Зарегистрироваться».
 - vi. Система сохраняет настройки и перенаправляет пользователя в Личный кабинет.
- **Альтернативный сценарий:** Введённый email уже зарегистрирован. На шаге 5 система подсвечивает поле email красным и выводит: «Пользователь с таким email уже существует».
- **Нежелательное завершение (ошибка соединения):** При нажатии кнопки регистрации (шаг 5) пропадает интернет. Система выводит уведомление «Ошибка сети. Проверьте подключение к интернету». Введённые данные сохраняются.
- **Отмена операции:** До шага 5 пользователь нажимает логотип компании в шапке сайта. Система возвращает его на главную страницу, процесс настройки прерывается.
- **Результат:** В базе данных создана новая учётная запись с привязанной ролью. Пользователь успешно авторизован.

2. Вход в систему

Действующее лицо: Зарегистрированный пользователь

Страницы: Главная → Вход

- **Предусловие:** Пользователь имеет зарегистрированный аккаунт, но в данный момент не авторизован.
- **Основной сценарий:**
 - i. Пользователь нажимает кнопку «Войти в аккаунт» на главной странице.
 - ii. Система открывает форму авторизации.
 - iii. Пользователь вводит свой email и пароль.
 - iv. Пользователь нажимает кнопку «Войти в аккаунт».
 - v. Система проверяет данные и перенаправляет пользователя на выбор роли (курьер, заказчик, администратор).
 - vi. Пользователь выбирает роль, система перенаправляет его в рабочую область (личный кабинет, панель курьера или дашборд администратора).
- **Альтернативный сценарий:** Пользователь ввёл неверный пароль. Система подсвечивает поле красным и отображает «Неверный логин или пароль». Вход блокируется до ввода верных данных.
- **Нежелательное завершение:** Аккаунт деактивирован администратором. При нажатии «Войти» система выдаёт: «Ваш аккаунт деактивирован. Обратитесь в службу поддержки»
- **Отмена операции:** Пользователь нажимает «Назад на главную» — система перенаправляет его на главную страницу.
- **Результат:** Пользователь авторизован в системе, установлена активная сессия.

3. Восстановление пароля

Действующее лицо: Зарегистрированный пользователь

Страницы: Экран авторизации → Экран восстановления

- **Предусловие:** Пользователь забыл пароль и находится на форме авторизации.
- **Основной сценарий:**
 - i. Пользователь нажимает ссылку «Забыли пароль?» в форме входа.
 - ii. Система отображает форму восстановления пароля.
 - iii. Пользователь вводит свой email и нажимает «Отправить код».
 - iv. Система отправляет письмо с кодом на указанный email и показывает поле для его ввода.
 - v. Пользователь вводит полученный код и новый пароль дважды.
 - vi. Пользователь нажимает «Сохранить».
 - vii. Система обновляет пароль и перенаправляет на форму входа.
- **Альтернативный сценарий:** Пользователь вводит email, которого нет в системе. Система выдаёт «Аккаунт с таким email не найден».
- **Нежелательное завершение:** Письмо с кодом попадает в спам или задерживается почтовым сервером — пользователь не может продолжить сценарий.
- **Отмена операции:** Пользователь нажимает «Назад» на любом этапе. Процесс сброса пароля отменяется.
- **Результат:** Пароль пользователя успешно изменён.

Теперь остановимся на сценариях использования заказчика:

1. Создание заказа на вывоз отходов

Действующее лицо: Клиент

Страницы: Заказчик → Мои заказы → Создать новый заказ

- **Предусловие:** Пользователь авторизован под ролью «Заказчик», находится в личном кабинете. На балансе достаточно средств.
- **Основной сценарий:**
 - i. Пользователь нажимает кнопку «Создать заказ».

- ii. Система открывает форму параметров заказа.
 - iii. Пользователь выбирает тип отходов (например, «Бытовой»).
 - iv. Пользователь вводит объём мусора.
 - v. Пользователь вводит адрес вывоза в текстовое поле.
 - vi. Пользователь выбирает желаемую дату и время вывоза.
 - vii. Пользователь нажимает «Оформить заказ».
 - viii. Система фиксирует заявку, холдирует средства с баланса и отображает уведомление с номером заказа.
- **Альтернативный сценарий:** Пользователь не указал адрес. При нажатии «Оформить заказ» система скроллит к пустому полю «Адрес», подсвечивает его красным с текстом «Это поле обязательно для заполнения».
 - **Нежелательное завершение:** Ошибка сохранения из-за сбоя в базе данных. Система выдаёт «Не удалось создать запись. Пожалуйста, повторите попытку позже».
 - **Отмена операции:** Пользователь нажимает «Мои заказы» в боковой панели. Форма закрывается без сохранения черновика.
 - **Результат:** В системе создан новый заказ со статусом «Поиск курьера». Заказ появляется в истории пользователя.

2. Пополнение баланса

Действующее лицо: Клиент

Страница: Пополнение баланса

- **Предусловие:** Пользователь авторизован и находится в разделе «Кошелёк» / «Баланс».
- **Основной сценарий:**
 - i. Пользователь нажимает «Пополнить баланс».
 - ii. Система открывает форму пополнения.

- iii. Пользователь вводит желаемую сумму (например, 1000) в числовое поле.
- iv. Пользователь выбирает метод оплаты (например, «Банковская карта»).
- v. Пользователь нажимает «Пополнить баланс».
- **Альтернативный сценарий:** На выбранном методе оплаты недостаточно средств. После шага 5 система возвращает ошибку «Отказ: недостаточно средств». Баланс не пополняется.
- **Нежелательное завершение:** В момент шага 5 теряется интернет-соединение. Пользователь видит системную ошибку сети. Статус платежа остаётся неизвестным до восстановления связи.
- **Отмена операции:** Пользователь переходит в другой раздел меню, не нажав «Пополнить баланс». Данные сбрасываются.
- **Результат:** Баланс кошелька пользователя увеличен на указанную сумму.

3. Управление профилем

Действующее лицо: Клиент

Страницы: Профиль

- **Предусловие:** Пользователь авторизован.
- **Основной сценарий:**
 - i. Пользователь переходит в раздел «Профиль».
 - ii. Система отображает текущие персональные данные.
 - iii. Пользователь изменяет данные в соответствующих полях.
 - iv. Пользователь нажимает «Сохранить изменения».
 - v. Система валидирует данные, обновляет БД и показывает «Данные успешно обновлены».

- **Альтернативный сценарий:** Пользователь ввёл телефон в некорректном формате. Система блокирует кнопку сохранения и выводит «Неверный формат номера».
- **Нежелательное завершение:** Internal Server Error при записи в БД. Система показывает «Не удалось сохранить данные. Попробуйте позже».
- **Отмена операции:** Пользователь вносит изменения, но уходит в другой раздел без сохранения. Изменения сбрасываются.
- **Результат:** Персональные данные пользователя обновлены в системе.

4. Просмотр истории заказов

Действующее лицо: Клиент

Страница: История заказов

- **Предусловие:** Пользователь авторизован под ролью «Заказчик» и имеет хотя бы один оформленный заказ.
- **Основной сценарий:**
 - Пользователь нажимает «История заказов» в боковом меню.
 - Система открывает страницу с таблицей всех оформленных заявок.
 - Пользователь применяет фильтры (по дате, типу мусора, статусу) или строку поиска.
 - Система обновляет список в соответствии с заданными параметрами.
- **Альтернативный сценарий:** У пользователя нет заказов или ничего не найдено по фильтрам. Система отображает «Заказы не найдены».
- **Нежелательное завершение:** Ошибка загрузки списка с сервера. Система показывает «Не удалось загрузить историю заказов. Попробуйте обновить страницу».

- **Отмена операции:** Пользователь переходит в другой раздел, не дожидаясь загрузки.
- **Результат:** Пользователь успешно просмотрел список своих заказов.

5. Просмотр деталей заказа

Действующее лицо: Клиент

Страница: Детали заказа

- **Предусловие:** Пользователь авторизован и находится на странице «История заказов».
- **Основной сценарий:**
 - i. Пользователь нажимает на строку заказа (например, №12345) или кнопку действий (три точки).
 - ii. Система открывает страницу «Детали заказа №12345».
 - iii. Пользователь просматривает: статус, дату/время, тип и объём отходов, адрес вывоза.
 - iv. Пользователь видит карточку курьера (фото, имя, рейтинг) и финансовый отчёт по стоимости услуги.
- **Альтернативный сценарий:** Заказ удалён администратором. При клике система выдаёт «Данный заказ не найден или был удалён».
- **Нежелательное завершение:** Потеря интернет-соединения при переходе. Страница деталей не загружается.
- **Отмена операции:** Пользователь нажимает «Назад к истории». Система возвращает его к таблице заказов.
- **Результат:** Пользователь ознакомился с подробной информацией о выбранном заказе.

6. Смена пароля

Действующее лицо: Клиент

Страницы: Личный профиль → Смена пароля

- **Предусловие:** Пользователь авторизован и находится в разделе «Профиль».
- **Основной сценарий:**
 - i. Пользователь нажимает «Сменить пароль» в форме личного профиля.
 - ii. Система открывает страницу (или модальное окно) «Смена пароля».
 - iii. Пользователь вводит текущий пароль в поле «Старый пароль».
 - iv. Пользователь вводит новый пароль (минимум 8 символов, заглавные буквы и цифры). Система визуально подтверждает выполнение требований.
 - v. Пользователь дублирует новый пароль в поле «Подтвердите новый пароль».
 - vi. Пользователь нажимает «Обновить пароль».
 - vii. Система проверяет правильность старого пароля и совпадение новых. При успехе сохраняет пароль, выводит уведомление и возвращает на страницу профиля.
- **Альтернативный сценарий:** Неверный старый пароль, несовпадение новых паролей или несоответствие требованиям безопасности. Система подсвечивает поля красным с соответствующим сообщением. Пароль не изменяется.
- **Нежелательное завершение:** Потеря интернет-соединения при нажатии «Обновить пароль». Данные не отправляются, пароль остаётся прежним.
- **Отмена операции:** Пользователь нажимает «Отмена». Форма закрывается, изменения не сохраняются.
- **Результат:** Пароль учётной записи успешно изменён.

Рассмотрим сценарии использования курьера

1. Просмотр и принятие заказа в работу

Действующее лицо: Курьер

Страницы: Доступные заказы → Взять заказ

- **Предусловие:** Курьер авторизован в системе, есть доступные заказы.
- **Основной сценарий:**
 - i. Курьер просматривает ленту «Доступные заказы», оценивая адрес, расстояние, тип отходов, время в пути и сумму оплаты.
 - ii. Курьер нажимает «Взять» в карточке интересующего заказа.
 - iii. Система открывает страницу «Детали заказа» с расширенной информацией: характеристики груза, точное местоположение, имя клиента, финансовые детали.
 - iv. Курьер нажимает «Взять заказ».
 - v. Система закрепляет заказ за курьером, убирает его из общего списка и переводит в раздел «Мои заказы».
- **Альтернативный сценарий:** Заказ перехвачен другим курьером. При нажатии «Взять заказ» система выводит «Заказ уже взят другим исполнителем» и возвращает к обновлённому списку.
- **Нежелательное завершение:** Потеря интернет-соединения при переходе в детали или нажатии кнопки. Заказ не закрепляется до восстановления связи.
- **Отмена операции:** Курьер перешёл в детали, но передумал. Нажимает «Назад» — система возвращает его к «Доступным заказам», статус заказа не меняется.
- **Результат:** Заказ закреплён за курьером, удалён из общего списка и переведён в работу.

2. Выполнение активного заказа

Действующее лицо: Курьер

Страницы: Мои заказы → Детали заказа

- **Предусловие:** Курьер имеет активный заказ в статусе «В пути».
- **Основной сценарий:**
 - i. Курьер переходит в «Мои заказы» и открывает карточку активного заказа.
 - ii. Курьер прибывает по адресу клиента и забирает отходы.
 - iii. Курьер нажимает «Заказ выполнен» в карточке заказа и прикрепляет фото выполненной работы.
 - iv. Система меняет статус на «Выполнен», уведомляет клиента, начисляет оплату на кошелёк курьера и обновляет статистику.
- **Альтернативный сценарий:** Клиент отменил заказ, пока курьер был в пути. На экране курьера появляется модальное окно «Заказ отменён клиентом», заказ переходит в статус отменённого.
- **Нежелательное завершение:** Потеря интернет-соединения при нажатии «Заказ выполнен». Статус заказа не обновляется до восстановления связи.
- **Отмена операции:** Курьер не может выполнить заказ по объективным причинам (клиента нет на месте, отходы не готовы). Курьер связывается с поддержкой. Статус меняется на «Отменён».
- **Результат:** Мусор вывезен. Статус заказа — «Выполнен». Клиент уведомлён, баланс и статистика курьера обновлены.

3. Просмотр кошелька, статистики и истории операций

Действующее лицо: Курьер

Страница: Кошелёк

- **Предусловие:** Курьер авторизован в системе и находится в панели управления.
- **Основной сценарий:**

- i. Курьер нажимает «Кошелёк» в боковом меню.
 - ii. Система загружает страницу «Кошелёк и статистика».
 - iii. Курьер просматривает виджеты: «Текущий баланс», «Заработано за месяц» (с динамикой), «Выполнено заказов» (за месяц и за всё время).
 - iv. Курьер прокручивает страницу до таблицы «История операций».
 - v. Курьер использует фильтры («За неделю», «За месяц», «За год»).
 - vi. Система обновляет список транзакций с датой, описанием и суммой (зелёный — пополнение, красный — списание).
- **Альтернативный сценарий:** Новый аккаунт без выполненных заказов. Система отображает нули в виджетах и «Операций пока нет» в таблице.
 - **Нежелательное завершение:** Сервер временно недоступен. Виджеты не загружаются, система показывает «Ошибка загрузки финансовых данных».
 - **Отмена операции:** Курьер переходит в другой раздел, не дожидаясь окончания загрузки.
 - **Результат:** Курьер ознакомился с балансом, статистикой и историей транзакций.

4. Управление профилем курьера

Действующее лицо: Курьер

Страницы: Профиль

- **Предусловие:** Пользователь авторизован под ролью курьера.
- **Основной сценарий:**
 - i. Курьер переходит во вкладку «Профиль» в боковом меню.

- ii. Система отображает текущие личные данные, рейтинг и информацию о транспорте.
 - iii. Курьер изменяет разрешённые данные в текстовых полях.
 - iv. Курьер нажимает «Сохранить изменения».
 - v. Система валидирует данные, обновляет БД и показывает уведомление об успехе.
- **Альтернативный сценарий:** Данные введены в неверном формате. Система блокирует сохранение и показывает подсказку об ошибке.
 - **Нежелательное завершение:** Ошибка сервера при записи в БД. Система выдаёт «Не удалось сохранить данные профиля. Попробуйте позже».
 - **Отмена операции:** Курьер уходит со вкладки без сохранения. Все изменения сбрасываются.
 - **Результат:** Персональные данные или данные о транспорте курьера обновлены в системе.

5. Смена пароля

Действующее лицо: Курьер

Страницы: Личный профиль → Смена пароля

- **Предусловие:** Курьер авторизован и находится в разделе «Профиль».
- **Основной сценарий:**
 - i. Курьер нажимает «Сменить пароль» в форме профиля.
 - ii. Система открывает страницу (или модальное окно) «Смена пароля».
 - iii. Курьер вводит текущий пароль в поле «Старый пароль».
 - iv. Курьер вводит новый пароль (минимум 8 символов, заглавные буквы и цифры). Система визуально подтверждает выполнение требований.

- v. Курьер дублирует новый пароль в поле «Подтвердите новый пароль».
- vi. Курьер нажимает «Обновить пароль».
- vii. Система проверяет старый пароль и совпадение новых. При успехе сохраняет пароль, выводит уведомление и возвращает на страницу профиля.
- **Альтернативный сценарий:** Неверный старый пароль, несовпадение или несоответствие требованиям. Система подсвечивает поля красным. Пароль не меняется.
- **Нежелательное завершение:** Потеря соединения при нажатии «Обновить пароль». Данные не отправляются, пароль остаётся прежним.
- **Отмена операции:** Курьер нажимает «Отмена». Форма закрывается без изменений.
- **Результат:** Пароль учётной записи курьера успешно изменён.

В завершение рассмотрим сценарии использования администратора:

1. Мониторинг систем

Действующее лицо: Администратор

Страница: Дашборд-админ

- **Предусловие:** Пользователь авторизован под учётной записью администратора.
- **Основной сценарий:**
 - i. Администратор открывает «Дашборд» через боковое меню.
 - ii. Система загружает сводную статистику (количество заказов, курьеры на линии).
 - iii. Администратор прокручивает страницу до блока «Лента событий».

- iv. Система отображает последние действия в реальном времени.
- v. Администратор нажимает «Загрузить предыдущие события».
- vi. Система подгружает историю логов за более ранний период.
- **Альтернативный сценарий:** Нет действий за текущий день. В ленте отображается «Событий за сегодня не найдено».
- **Нежелательное завершение:** Одно из событий в ленте содержит ссылку на удалённый объект (заказ или пользователь). Строка не рендерится, рядом выводится: «Ошибка отображения события: связанный объект не найден». Остальная лента загружается корректно.
- **Отмена операции:** Во время подгрузки (шаг 5) администратор переходит в другой раздел. Подгрузка прерывается.
- **Результат:** Администратор получил актуальную информацию о состоянии системы и просмотрел логи действий.

2. Создание резервной копии

Действующее лицо: Администратор

Страницы: Дашборд-админ, Пользователи-админ, Заказы-админ

- **Предусловие:** Администратор авторизован и находится в панели управления. В БД есть сохранённые данные.
- **Основной сценарий:**
 - i. Администратор переходит к блоку «Управление данными».
 - ii. Администратор нажимает «Скачать дамп (Экспорт JSON)».
 - iii. Система запускает формирование резервной копии всех коллекций БД.
 - iv. Система отдаёт файл .json браузеру.
 - v. Администратор сохраняет файл на жёсткий диск через стандартное окно ОС.

- **Альтернативный сценарий:** БД слишком велика, время генерации превышено. Система выдаёт «Ошибка генерации: превышен лимит времени».
- **Нежелательное завершение:** Обрыв скачивания из-за нестабильного интернета. Загружается неполный («битый») JSON-файл.
- **Отмена операции:** Администратор нажимает «Отмена» в системном диалоге браузера при сохранении файла (шаг 5).
- **Результат:** Полная резервная копия БД успешно сохранена локально у администратора.

3. Восстановление данных из резервной копии

Действующее лицо: Администратор

Страницы: Дашборд-админ, Пользователи-админ, Заказы-админ

- **Предусловие:** Администратор авторизован. На локальном устройстве есть валидный файл резервной копии в формате JSON.
- **Основной сценарий:**
 - Администратор переходит к блоку «Управление данными».
 - Администратор нажимает «Загрузить дамп (Импорт JSON)».
 - Система открывает стандартное окно ОС для выбора файла.
 - Администратор выбирает файл .json и подтверждает выбор.
 - Система загружает файл на сервер, валидирует его структуру и начинает запись данных в ArangoDB.
 - По завершении система выводит «Данные успешно восстановлены» и обновляет статистику на экране.
- **Альтернативный сценарий:** Выбран файл неверного формата или повреждённый JSON. На шаге 5 система прерывает загрузку и выводит «Ошибка формата: выберите корректный файл дампа базы данных».

- **Нежелательное завершение:** Разрыв соединения или сбой сервера во время записи (шаг 5). Система выдаёт «Критическая ошибка при восстановлении. Часть данных может быть утеряна».
- **Отмена операции:** Администратор нажимает «Отмена» в окне выбора файла (шаг 4). Окно закрывается, загрузка не инициируется.
- **Результат:** Данные в ArangoDB успешно перезаписаны или дополнены из файла резервной копии.

4. Управление заказами

Действующее лицо: Администратор

Страница: Заказы на вывоз

- **Предусловие:** Администратор авторизован в системе и находится в разделе «Заказы на вывоз».
- **Основной сценарий:**
 - Администратор просматривает общий список заказов в таблице (ID, даты, клиент, курьер, статус).
 - Администратор обращается к блоку «Многокритериальный фильтр».
 - Администратор вводит ID заказа, выбирает «Статус» или «Тип мусора» из выпадающих списков.
 - Администратор нажимает «Применить фильтр».
 - Система фильтрует БД и обновляет таблицу.
- **Альтернативный сценарий:** По заданным критериям нет совпадений. Система оставляет таблицу пустой с сообщением «Заказы не найдены».
- **Нежелательное завершение:** Фильтр возвращает слишком большой набор данных. Система обрезает результат и предупреждает: «Показаны первые 1000 записей. Уточните параметры фильтрации».

- **Отмена операции:** Администратор очищает поля фильтра и снова нажимает «Применить фильтр», возвращаясь к полному списку заказов.
- **Результат:** Администратор успешно нашёл и отфильтровал нужные заказы.

5. Управление пользователями

Действующее лицо: Администратор

Страница: Пользователи

- **Предусловие:** Администратор авторизован и перешёл во вкладку «Пользователи» через боковое меню.
- **Основной сценарий:**
 - Администратор просматривает таблицу всех зарегистрированных пользователей (имя, контакты, роль, статус аккаунта).
 - Для поиска конкретного пользователя администратор использует «Многокритериальный фильтр».
 - Администратор вводит ФИО, Email или телефон в поле поиска, либо выбирает «Роль в системе» и «Статус аккаунта».
 - Администратор нажимает «Применить фильтр».
 - Система обновляет список в соответствии с фильтрами.
- **Альтернативный сценарий:** По критериям фильтрации нет совпадений. Система оставляет таблицу пустой с сообщением «Пользователи не найдены».
- **Нежелательное завершение:** Сбой сервера при применении фильтра. Система выдаёт «Ошибка загрузки данных заказа. Пожалуйста, обновите страницу».
- **Отмена операции:** Администратор очищает поля фильтра и снова нажимает «Применить фильтр», возвращаясь к полному списку.

- **Результат:** Администратор успешно отфильтровал список пользователей и нашёл нужный профиль.

3. МОДЕЛЬ ДАННЫХ

3.1 Нереляционная модель данных

В разработанной модели данных(рис.1) система использует следующие коллекции:

- Users
- Orders
- Locations
- Transactions

Связи в ArangoDB реализуются с помощью специальных сущностей-ребер:

- Owns
- Executes
- At
- History

Также можно организовать связи по ключам, подобно sql, однако было решено использовать именно edge collections по следующим причинам:

1. Этот вид связи куда нагляднее демонстрирует возможности графовой БД, что в учебном проекте важнее максимальной эффективности.
2. Если же говорить о рациональности, то использование edge collections делает систему более гибкой. Если в будущем теоретически возникнет необходимость хранить какую-нибудь дополнительную информацию о связях, то ее можно будет записать именно в атрибуты ссылок-документов, а не придумывать дополнительные сущности или изменять основные коллекции.

3. Также встроенные эффективные алгоритмы графовых обходов могут пригодиться при потенциальном расширении. Условно, операция "найти всех клиентов, которых обслужил данный курьер за неделю" может быть реализована через BFS эффективнее, чем в случае связи по `_id`.
4. Лучшая "читаемость". Вершины - существительные. Ребра - действия. Лишь взглянув на диаграмму БД с edge-collections, можно понять, как устроена база данных. Отдельно отметим удобство связей Owns и Executes. Обе идут от заказа к пользователю, но смысл у них разный.

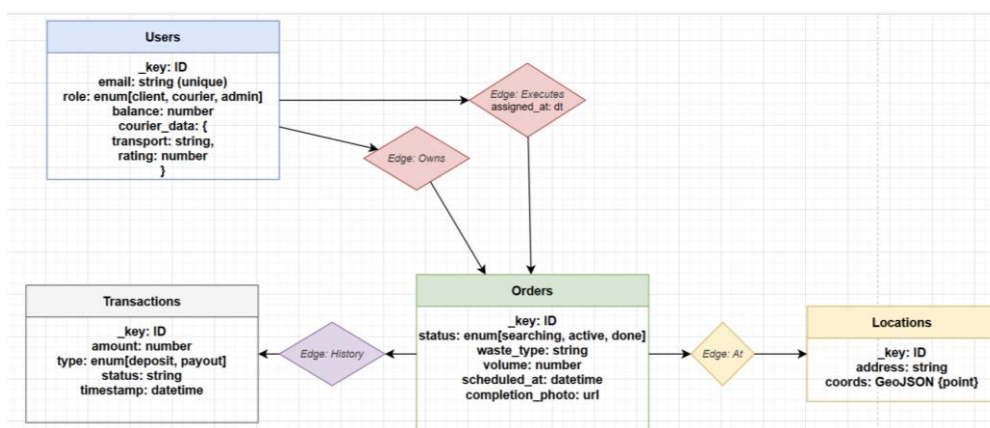


Рисунок 1. Нереляционная модель данных

В данной модели данных используются следующие коллекции:

- *Users* - хранит необходимые данные клиента/курьера
- *Orders* - хранит все данные о конкретном заказе
- *Locations* - хранит координаты локации для каждого заказа
- *Transactions* - записи о движении денежных средств (депозиты, выплаты, статусы платежей).
- *Owns* (*Users* → *Orders*) — связывает клиента с его заявками.
- *Executes* (*Users* → *Orders*) — связывает курьера с принятыми в работу заказами.

- *At* (Orders → Locations) — указывает, где физически должен быть выполнен (или был выполнен) заказ.
- *History* (Orders → Transactions) — связывает выполненный заказ с финансовым отчетом. Позволяет отследить оплату по каждой конкретной услуге.

Примеры хранения данных в модели БД.

Рассмотри на примере: клиент заказал вывоз мусора из конкретного места, а курьер выполнил заказ.

- Users(Рис.2)

```
// Пример: Клиент (0.9N)
{
  "_key": "user_1",
  "_id": "Users/user_1",
  "email": "client@example.com",
  "password_hash": "xxxxxx...",
  "name": "Ivan Ivanov",
  "phone": "+79001112233",
  "role": "client",
  "balance": 1500.50
}

// Пример: Курьер (0.1N)
{
  "_key": "courier_1",
  "role": "courier",
  "courier_data": {
    "transport": "bicycle",
    "rating": 4.95,
    "is_active": true
  }
}
```

Рисунок 2. Пример хранения данных курьера и заказчика

- Order(Рис.3)

```
{
  "_key": "ord_1",
  "waste_type": "plastic",
  "status": "completed",
  "volume": 12.5,
  "created_at": 111111,
  "completed_at": 222222,
  "completion_photo": "[https://static.service.io/shards/a7/photo_555.jpg]"
}
```

Рисунок 3. Пример хранения данных заказа

- Location(Рис.4)

```
{
  "_key": "loc_spb_nev",
  "address": "г. Санкт-Петербург, Невский проспект, д. 50",
  "coords": {
    "type": "Point",
    "coordinates": [0.0, 0.0]
  }
}
```

Рисунок 4. Пример хранения данных локации

- Transaction(Рис.5)

```
{
  "_key": "tx_1",
  "amount": 450.0,
  "type": "order_payout",
  "status": "success",
  "timestamp": 222222,
  "external_id": "PAYPAL-TX-998877665544-RECONCILE"
}
```

Рисунок 5. Пример хранения данных транзакции

Набор сущностей вершин в ArangoDB должен быть связан сущностями-ребрами, которые можно трактовать как "глаголы":

- Owns(Клиент - Заказ)(Рис.6)

```
{
  "_key": "edge_owns_101",
  "_id": "Owns/edge_owns_101",
  "_from": "Users/user_client_1",
  "_to": "Orders/ord_1"
}
```

Рисунок 6. Пример связи между клиентом и заказом

- Executes(Курьер - Заказ)(Рис.7)

```
{
  "_key": "edge_exec_202",
  "_id": "Executes/edge_exec_202",
  "@@ -247,134 +198,166 @@ k = 14343N / 3045N = **4.71**"
}
```

Рисунок 7. Пример связи между курьером и заказом

- At(Заказ - Локация)(Рис.8)

```
{
  "_key": "edge_at_303",
  "_id": "At/edge_at_303",
  "_from": "Orders/ord_1",
  "_to": "Locations/loc_spb_nev"
}
```

Рисунок 8. Пример связи между заказом и локацией

- History(Заказ - Транзакция)(Рис.9)

```
{
  "_key": "edge_hist_404",
  "_id": "History/edge_hist_404",
  "_from": "Orders/ord_1",
  "_to": "Transactions/tx_1"
}
```

Рисунок 9. Пример связи между заказом и транзакцией

Размеры полей

1. Системный Overhead (Метаданные: 100 байт)

Каждый документ в ArangoDB (Vertex или Edge) имеет метаданные, обусловленную местными особенностями в лице формата VPack и движка RocksDB:

- `_key`: Строковый уникальный идентификатор (10–20 байт).
- `_id`: Полный путь Collection/Key (~30 байт).
- `_rev`: Системная ревизия документа (8–16 байт).
- *VPack Headers*: Таблицы смещений (Offsets) для быстрого доступа к полям без полного парсинга JSON (~20-30 байт).

2. Ребра (Edges: 200 байт)

Ребро — это документ, содержащий две обязательные индексируемые ссылки:

- *_from + _to*: Строковые ссылки на ID вершин. Каждая занимает ~40-50 байт.
- *Метаданные*: Стандартные 100 байт. Таким образом, даже «пустое» ребро весит около 200 байт.

-

3. Взвешенная оценка пользователя (User: 292 байта)

Используется средневзвешенное значение, так как *courier_data* присутствует только у 10% записей(поскольку мы условились, что курьеров - 10% от всех пользователей):

- *Auth (Email/Pass)*: 100 байт (хэш пароля + строка email).
- *Profile (Name/Phone)*: 60 байт (UTF-8 строки).
- *Balance/Role*: 20 байт (Numeric типы в VPack компактны).
- *Courier Data*: 120 байт * 0.1 = 12 байт.

4. Оценка заказа (Order: ~500 байт)

- *Metadata*: 100 байт.
- *Type + Status*: 40 байт (тип отходов и текущий этап заказа).
- *Volume + Timestamps*: 60 байт (объем в числовом формате и 2-3 метки времени: создание, назначение, финализация).
- *Completion_photo (URL)*: 200 байт (ссылка на CDN-хранилище с подтверждением выполнения).
- *Business Flags*: 50 байт (дополнительные булевы значения: "опасные отходы", "нужна доп. помощь" и т.д.).

5. Оценка локации (Location: ~300 байт)

- *Metadata*: 100 байт.
- *Address String*: 120 байт (нормализованная строка адреса для отображения пользователю).

- *Coords (GeoJSON)*: 80 байт (структура {"type": "Point", "coordinates": [lat, lon]}).
-
- 6. Оценка транзакции (Transaction: ~350 байт)
- *Metadata*: 100 байт.
- *Financial Details*: 50 байт (сумма в float64, тип операции и внутренний статус).
- *Timestamp*: 16 байт (точное время проведения транзакции).
- *External Reference*: 180 байт (ID транзакции во внешнем шлюзе и описание для банковской выписки).

Оценка объема данных

Пусть N — общее число пользователей сервиса. Предположим распределение ролей и активности объектов следующим:

- *Клиенты*: $0.9N$ (у каждого по 5 заказов).
- *Курьеры*: $0.1N$ (у каждого в среднем по 45 выполненных заказов).
- *Заказы (O)*: $0.9N * 5 = 4.5N$.
- *Локации*: $4.5N$
- *Транзакции (T)*: $4.5N * 1.2 = 5.4N$ (с учетом выплат и пополнений).
- *Ребра (E)*: Сумма всех связей (Owns, Executes, At, History) = $18.9N$.

Рассчитаем вес всех сущностей. Для расчетов используется внутренняя спецификация формата VPack, применяемого в ArangoDB(табл.1).

Таблица 1. Веса сущностей

Сущность	Чистый вес (байт)	Метаданные (байт)	Итого V
User (Vu)	192	100	292
Order (Vo)	400	100	500

Location (Vl)	200	100	300
Transaction (Vt)	250	100	350
Edge (Ve)	100 (_from/_to)	100	200

Суммарный объем просто данных в БД:

$$S(N) = N * 292 + 4.5N * 500 + 4.5N * 300 + 5.4N * 350 + 18.9N * 200 \\ = 9562N \text{ Байт}$$

Для обеспечения высокой скорости графовых обходов (Traversals) и гео-запросов, ArangoDB строит:

1. *Primary Index* (Hash-таблица по _key).
2. *Edge Index* (Списки смежности для каждой вершины).
3. *Geo/Persistent Indexes* (Для поиска по координатам и email).

По практике эксплуатации, индексы занимают ~50% от объема данных.

Финальная формула фактического объема:

$$S_full(N) = S(N) * 1.5 = 14343N \text{ Байт.}$$

Для вычисления избыточности определим *Полезный объем* (S_useful) — только бизнес-данные без учета связей, индексов и чисто технических полей.

- *Users*: 192 байта.
- *Orders*: 350 байт.
- *Locations*: 200 байт.
- *Transactions*: 70 байт.
- *Edges*: 0 байт.

$$S_useful(N) = N * 192 + 4.5N * 350 + 4.5N * 200 + 5.4N * 70 = 3045N \text{ Байт}$$

Рассчитаем коэффициент избыточности: $k = 14343N / 3045N = 4.71$

Итог: на каждый 1 байт полезной бизнес-информации система резервирует 3.71 байта для обеспечения целостности связей, возможности

быстрого поиска и расширяемости модели. Как видно, избыточность растет линейно относительно N .

Направление роста модели при масштабировании

В графовых БД рост может характеризоваться как обычным увеличением узлов, так и часто следующим за этим увеличением локальной плотности связей и степени вершины. Теоретически рассмотрим сценарии при увеличении числа объектов каждой из сущностей.

1. Users — Горизонтальный рост

Рост этой сущности является *базовым вектором*.

- *Характер:* Линейный ($O(N)$).
- *Влияние:* Увеличение количества пользователей заставляет граф расти вширь, без уплотнения связей и появления супервершин. Это самый предсказуемый тип роста, который легко компенсируется горизонтальным масштабированием по первичному ключу `_key`.

2. Orders — Плотность и Вертикальный рост

Заказы генерируются пользователями, и их количество растет быстрее, чем число самих пользователей.

- *Характер:* Накопительный. Со временем среднее число заказов на одного клиента может вырасти с 5 до 50+.
- *Влияние:* Это создает "*Супер-узлы*" — объекты с большим количеством связей. Если один курьер выполнит тысячи заказов, ребро `Executes` станет "тяжелым", что замедлит графовые обходы (Traversals) конкретно для этого узла.
- *Направление:* Уплотнение связей вокруг активных профилей.

3. Transactions

- *Характер*: Поточковый. На каждый успешный заказ приходится минимум 1.2 транзакции.
- *Влияние*: Рост этой коллекции наиболее существенно влияет на *объем дискового пространства*. Поскольку транзакции редко участвуют в сложных графовых обходах (обычно это поиск User -> Transaction), они не замедляют логику сервиса

4. Locations — Географическая кучность

Локации привязаны к физическим координатам.

- *Характер*: Пространственный.
- *Влияние*: В отличие от других сущностей, локации растут *кластерами* (например, высокая плотность в центре мегаполиса и низкая на окраинах).
- *Направление*: Нагрузка на *Geo-Index*. При увеличении количества заказов в одном районе дерево гео-индекса становится глубже, что может незначительно увеличить время поиска ближайших курьеров в этой зоне.

Примеры запросов

1. Регистрация и авторизация

а. Запрос регистрации

```
INSERT {
  "email": @email,
  "password_hash": @hash,
  "name": @name,
  "phone": @phone,
  "role": @role,
  "balance": 0,
  "courier_data": @courier_data
} INTO Users
RETURN NEW._id
```

- Количество запросов 1 на каждую регистрацию
- Коллекции Users - 1

b. Запрос авторизации

```
FOR u IN Users
  FILTER u.email == @email
  LIMIT 1
  RETURN u
```

- Количество запросов 1 на каждую авторизацию
- Коллекции Users - 1

2. Клиентские операции

a. Создание заказа

```
// 1. Создание заказа
LET order = (INSERT @order_data INTO Orders RETURN NEW)[0]
// 2. Создание локации
LET loc = (INSERT @loc_data INTO Locations RETURN NEW)[0]
// 3. Привязка владельца
INSERT { _from: @user_id, _to: order._id } INTO Owns
// 4. Привязка локации
INSERT { _from: order._id, _to: loc._id } INTO At
```

- Количество запросов: 4 на каждый заказ.
- Коллекций: 4 (Orders, Locations, Owns, At).

b. История заказов

```
FOR order IN 1..1 OUTBOUND @user_id Owns
  FILTER order.status == @status || @status == null
  SORT order.created_at DESC
  RETURN order
```

- Количество запросов: 1 на каждый просмотр истории.
- Коллекций: 2

3. Курьерские операции

a. Взять заказ в работу

```
// Смена статуса заказа
UPDATE @order_id WITH { status: "in_progress" } IN Orders
// Создание связи исполнения
INSERT { _from: @courier_id, _to: @order_id,
"assigned_at": DATE_NOW() } INTO Executes
```

- Количество запросов: 2 на каждый принятый заказ.
- Коллекций: 2 (Orders, Executes).

b. Завершение заказ и оплата

```
// 1. Обновляем заказ
UPDATE @order_id WITH { status: "completed",
"completion_photo": @url } IN Orders
// 2. Создаем транзакцию
```

```

LET tx = (INSERT @tx_data INTO Transactions RETURN NEW)[0]
// 3. Связываем заказ с транзакцией
INSERT { _from: @order_id, _to: tx._id } INTO History
// 4. Начисляем баланс
UPDATE @courier_id WITH { balance: balance + @amount } IN
Users

```

- Количество запросов: 4 на успешное завершение.
- Коллекций: 4 (Orders, Transactions, History, Users).

4. Административные запросы

a. Мониторинг системы

```

RETURN {
  total_orders: LENGTH(Orders),
  active_couriers: LENGTH(FOR u IN Users FILTER u.role
== "courier" AND u.is_online == true RETURN 1),
  pending_orders: LENGTH(FOR o IN Orders FILTER o.status
== "pending" RETURN 1)
}

```

- Количество запросов: 1
- Коллекций: 2 (Orders, Users).

b. Фильтр заказов по критериям

```

FOR o IN Orders
  FILTER (@status == null || o.status == @status)
  FILTER (@type == null || o.waste_type == @type)
  LIMIT @offset, @limit
RETURN o

```

- Кол-во запросов: 1 на каждое действие фильтрации.
- Коллекций: 1 (Orders).

3.2. Реляционная модель данных

В реляционной модели сущности-ребра превращаются либо в Foreign Keys (для связей "один ко многим"), либо в Join Tables (для связей "многие ко многим"):

- Users (Вершина)
- Orders (Центральная сущность)
- Locations (Сущность адресов)
- Transactions (Финансовая сущность)

Рассмотрим таблицы связей, которые были использованы а реляционной модели данных(Рис.10):

- *Users*: Хранит учетные данные. Роль (role) определяет, является ли пользователь клиентом или курьером.
- *Orders*: Основная таблица. Содержит client_id (FK к Users) и courier_id (FK к Users, nullable). Это заменяет ребра Owns и Executes.
- *Locations*: Хранит географические данные. Связана с заказом через location_id в таблице Orders.
- *Transactions*: Финансовые записи. Содержит order_id (FK к Orders), что заменяет ребро History.

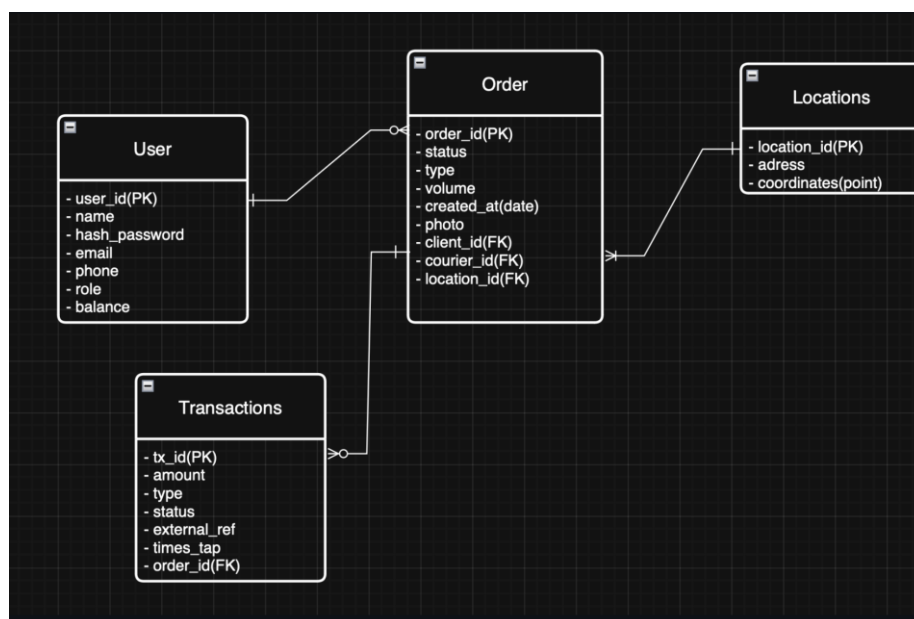


Рисунок 10. Реляционная модель данных

Оценка объёма

При тех же вводных (N пользователей, $4.5N$ заказов), веса строк в SQL будут меньше за счет типизации(табл.2).

Таблица 2 - объем модели

Таблица	Чистый вес (байт)	Индексы/Header (байт)	Итого (байт)
Users	150	50	200

Orders	250	100	350
Transactions	100	40	140
Locations	120	40	160

Фактический объем данных без индексов:

$$S_data(N)=N \cdot 200 + 4.5N \cdot 350 + 4.5N \cdot 160 + 5.4N \cdot 140 = 3251N \text{ байт.}$$

В реляционных базах индексы (B-Tree) обычно занимают ~30% от объема таблицы (в отличие от 50% в графовых, где индексируются все связи-ребра).

Итоговый фактический объем:

$$S_full(N)=3251N \cdot 1.3 \approx 4226N \text{ байт.}$$

Избыточность данных

Полезный объем (S_useful): Берем только «чистые» бизнес-данные (те же значения, что вы использовали для нереляционной модели, для честного сравнения).

$$S_useful(N)=N \cdot 192 + 4.5N \cdot 350 + 4.5N \cdot 200 + 5.4N \cdot 70 = 3045N \text{ байт.}$$

Коэффициент избыточности (k):

$$k \approx 1.39$$

Вывод: В реляционной модели избыточность значительно ниже (1.39 против 4.71 в ArangoDB). Это связано с тем, что SQL не хранит названия полей в каждой строке и не создает отдельные документы для каждой связи.

Направление роста модели

В реляционной модели рост сущностей влияет на производительность иначе: здесь нет проблемы «супер-узлов», но есть проблема глубины индексов и стоимости JOIN-операций.(табл.3)

Таблица 3 - направление роста модели

Таблица	Тип роста	Главный ресурс	Влияние на систему
Users	Линейный	RAM (Индексы)	Стабильный рост. При достижении миллионов записей замедляется поиск по email.
Orders	Интенсивный	Диск, (поиск)	Рост таблицы увеличивает стоимость JOIN с транзакциями и локациями.
Transactions	Потоковый	Диск	Самая быстрорастущая таблица. Влияет на время создания бэкапов.
Locations	Геометрический	CPU	Увеличение количества точек в GiST индексе замедляет расчеты «ближайшего курьера».

Рассмотри примеры данных в реляционной модели данных:

1. Users

```
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    role VARCHAR(20) CHECK (role IN ('client', 'courier')),
    balance DECIMAL(12, 2) DEFAULT 0
);
```

2. Orders

```
CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    client_id INT REFERENCES users(user_id),
```

```

    courier_id INT REFERENCES users(user_id),
    location_id INT REFERENCES locations(location_id),
    status VARCHAR(50),
    waste_type VARCHAR(50),
    completion_photo_url TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    assigned_at TIMESTAMP
);

```

3. Locations

```

CREATE TABLE locations (
    location_id SERIAL PRIMARY KEY,
    address TEXT NOT NULL,
    coordinates POINT -- Гео-тип
);

```

4. Transactions

```

CREATE TABLE transactions (
    tx_id SERIAL PRIMARY KEY,
    order_id INT REFERENCES orders(order_id),
    amount DECIMAL(12, 2),
    status VARCHAR(20),
    external_ref TEXT
);

```

Рассмотрим примеры запросов:

1. Запрос для регистрации

```

INSERT INTO users (email, password_hash, name, phone, role, balance)
VALUES (:email, :hash, :name, :phone, :role, 0)
RETURNING user_id;

```

- Количество запросов: 1 на регистрацию.
- Таблицы: users.

2. Запрос авторизации

```

SELECT * FROM users
WHERE email = :email
LIMIT 1;

```

- Количество запросов: 1 на вход.
- Таблицы: users.

3. Запрос для создания заказа

```

BEGIN;
-- 1. Создаем локацию

```

```

INSERT INTO locations (address, coordinates)
VALUES (:address, ST_GeomFromText(:point))
RETURNING location_id; -- Допустим, получили ID = 10
-- 2. Создаем заказ, сразу привязывая клиента и локацию (FK)
INSERT INTO orders (client_id, location_id, waste_type, status,
volume)
VALUES (:user_id, 10, :waste_type, 'pending', :volume);
COMMIT;

```

- Количество запросов: 2 (или 1 транзакция с двумя INSERT).
- Таблицы: locations, orders.

4. Запрос для истории заказов

```

SELECT o.*, l.address
FROM orders o
JOIN locations l ON o.location_id = l.location_id
WHERE o.client_id = :user_id
AND (:status IS NULL OR o.status = :status)
ORDER BY o.created_at DESC;

```

- Количество запросов: 1.
- Таблицы: locations, orders.

5. Запрос для того, чтобы взять заказ в работу

```

UPDATE orders
SET status = 'in_progress',
    courier_id = :courier_id,
    assigned_at = NOW()
WHERE order_id = :order_id AND status = 'pending';

```

- Количество запросов: 1.
- Таблицы: orders.

6. Завершение заказа и оплата

```

BEGIN;
-- 1. Обновляем статус заказа
UPDATE orders SET status = 'completed', completion_photo_url = :url
WHERE order_id = :order_id;
-- 2. Создаем транзакцию (связь через order_id в самой таблице
транзакций)
INSERT INTO transactions (order_id, amount, type, status) VALUES
(:order_id, :amount, 'payout', 'success');
-- 3. Начисляем баланс
UPDATE users SET balance = balance + :amount WHERE user_id =
:courier_id;
COMMIT;

```

- Количество запросов: 3.
- Таблицы: orders, transactions, users.

7. Запрос для мониторинга системы

```
SELECT
    (SELECT COUNT(*) FROM orders) AS total_orders,
    (SELECT COUNT(*) FROM users WHERE role = 'courier' AND is_online
= true) AS active_couriers,
    (SELECT COUNT(*) FROM orders WHERE status = 'pending') AS
pending_orders;
```

- Количество запросов: 1.
- Таблицы: orders, users.

8. Запрос с использованием фильтров заказов

```
SELECT * FROM orders
WHERE (:status IS NULL OR status = :status)
    AND (:type IS NULL OR waste_type = :type)
ORDER BY created_at DESC
LIMIT :limit OFFSET :offset;
```

- Количество запросов: 1.
- Таблицы: orders.

3.3 Сравнение моделей

При прочих равных исходных данных, *нереляционная модель (ArangoDB)* требует значительно больше места на диске и в оперативной памяти. В документно-графовых хранилищах удельный объем выше из-за:

1. Хранения имен ключей вместе со значениями каждого документа (JSON).
2. Выделения связей в отдельные коллекции ребер (Owns, Executes), каждое из которых имеет мета-поля.
3. Огромного количества метаданных на уровне СУБД (например, ключи ревизий, _key, _id).

Рассмотрим запросы по отдельным usecase(табл.4)

Таблица 4. запросы по отдельным usecase

Usecase	Запросов (СУБД) (NoSQL)	Запросов (SQL)	Коллекций (NoSQL)	Таблиц (SQL)
Регистрация	1	1	1	1
Создание заказа	4	2	4	2
Курьер берет заказ	2	1	2	1
Завершение заказа	4	3	4	3
Выборка истории	1	1	2	2

Исходя из рассмотренной предметной области (сервис по вывозу мусора, геолокационная маршрутизация, гибкие требования к заказам):

1. *Реляционная (SQL) модель* выигрывает с точки зрения экономии памяти и подходит для строго структурированных данных. Однако при масштабировании, добавлении сложных гео-запросов и быстром изменении структуры атрибутов (например, внедрении новых типов отходов, услуг или специфичных параметров ТС) она становится менее гибкой и требует регулярных ресурсоемких миграций схемы (Schema Migrations).
2. *Нереляционная графово-документная модель (ArangoDB)* дает существенную гибкость схем данных, нативную поддержку пространственных индексов (GeoJSON) для эффективного поиска заказов рядом с курьерами и мощный графовый движок для анализа связей. В условиях сервиса, где требуется быстро связывать различные сущности и хранить разнородные детали заявок,

документоориентированный подход полностью оправдывает накладные расходы на хранение. Кроме того, графовый обход обеспечивает предсказуемую скорость выборки истории $O(K)$ независимо от размера базы N .

Итог: Для данного проекта более предпочтительным решением является *нереляционная база данных (ArangoDB)*. Ее архитектура (документы + графы + Geo) идеально закрывает ключевые бизнес-потребности: эффективную работу с геопозицией, высокую скорость разработки за счет schema-less подхода и возможность горизонтального масштабирования при росте числа пользователей и заказов.

3.4. Сравнительный анализ теоретических и практических объёмов хранения данных

Согласно теоретической модели, структура графа базы данных линейно зависит от общего числа пользователей N . При тестировании системы на масштабе MEDIUM общее число пользователей составило $N = 400$ (из которых 360 — заказчики, а 40 — курьеры, что строго соответствует пропорции 90% на 10%).

В таблице 5 представлено сравнение теоретических оценок количества документов с реальными результатами, полученными из логов СУБД ArangoDB.

Таблица 5. Сравнение теоретического и практического состава документов

Наименование коллекции	Тип коллекции	Теоретическая оценка ($N=400$)	Практический результат (Seed)	Анализ причин расхождения
Users	Документы	400	400	Базовая опорная точка

				масштабирования системы.
Orders	Документы	$4.5N = 1800$	1800	Идеальное совпадение (ровно 5 заказов на каждого из 360 клиентов).
Addresses	Документы	$4.5N = 1800$	1800	Синхронный рост вслед за сущностью «Заказ» (строгая связь 1:1).
Transactions	Документы	$5.4N = 2160$	1966	Часть созданных заказов еще не дошла до этапа финальной оплаты или выплаты курьеру.
Все ребра графа(At, Owns, Executes, History)	Ребра	$18.9N = 7560$	6907	Обусловлено тем, что часть заказов находится в статусах поиска и не успела породить ребра исполнителей.

Также во время написания кода была добавлена ещё одна коллекция Events, которая на практике заняла 1966 байт.

Для сопоставления объемов памяти чистые бизнес-данные (полезный объем S_{useful}) были очищены от системных метаданных, связей и индексов.

Используя реальное количество документов из таблицы 1 и точные весовые характеристики сущностей (включая системный оверхед бинарного формата VelocityPack в 100 байт на документ и 200 байт на ребро), был пересчитан фактический физический объем памяти. При расчете полного объема учитывалось место под индексы (Primary, Edge и Geo) с коэффициентом 1.5. Итоговое сравнение теоретических и практических показателей избыточности представлено в таблице 6.

Таблица 6. Сравнительный анализ показателей избыточности данных

Метрика объема	Теоретическая модель	Практический срез (LARGE)	Формула расчета / Обоснование
Полезный объем (S_{useful})	3045N Байт	1 342 040 Байт	Чистый вес бизнес-полей (без метаданных СУБД, ребер и с учетом коллекции Events).
Полный объем базы (S_{full})	14343N Байт	6 471 600 Байт	Суммарный физический объем (данные документов + ребра + 50% на индексы).
Коэффициент избыточности (k)	4.71	4.82	Коэффициент избыточности рассчитывался, как отношение S_{full} к S_{useful}

Анализ изменения коэффициента избыточности:

Практический коэффициент избыточности данных ($k = 4.82$) оказался крайне близок к теоретическому прогнозу ($k = 4.71$), отклонение составило всего 2.3%. Небольшой рост избыточности на практике объясняется введением в систему новой служебной коллекции Events (1966 документов), которая добавила дополнительный инфраструктурный оверхед на хранение метаданных RocksDB (`_key`, `_id`, `_rev`).

Проведенный сравнительный анализ подтвердил высокую точность разработанной теоретической модели: погрешность прогнозирования итогового объема дисковой подсистемы не превысила 3%.

Эксперимент на масштабе *MEDIUM* наглядно демонстрирует специфику графовых NoSQL-решений: для обеспечения быстрой связности и эффективного выполнения AQL-запросов система тратит на хранение индексов и бинарных заголовков VelocityPack почти в 5 раз больше места, чем весят чистые бизнес-данные ($k = 4.82$).

4. РАЗРАБОТКА ПРИЛОЖЕНИЯ

4.1. Краткое описание

Система построена на базе контейнеризированной архитектуры, что обеспечивает изолированность компонентов и простоту развертывания. Взаимодействие между фронтендом и бэкендом осуществляется по протоколу REST API с использованием JWT-авторизации.

Контейнерная структура (Docker)

Проект разделен на три основных сервиса, запускаемых в изолированных Docker-контейнерах:

- *Frontend-контейнер*: SPA-приложение на базе *React* и *Vite*. Отвечает за интерфейс пользователя (заказчика, курьера и администратора) и взаимодействие с API.

- *Backend-контейнер*: Приложение на *FastAPI (Python)*. Содержит бизнес-логику, эндпоинты для обработки заказов, систему аутентификации и интеграцию с базой данных.
- *Database-контейнер*: NoSQL СУБД *ArangoDB*. Хранит данные о пользователях, коллекцию заказов и историю событий в виде документов.

Модульная структура Backend

Бэкенд организован по принципу разделения ответственности (Separation of Concerns):

- *Core*: Конфигурация системы (*Dynaconf*), настройки безопасности (JWT) и глобальные константы.
- *API (Routes)*: Модули, определяющие эндпоинты для каждой роли:
 - *auth*: Регистрация и вход.
 - *clients*: Личный кабинет заказчика и управление профилем.
 - *courier*: Логика обработки заявок курьером.
 - *admin*: Панель мониторинга и управление пользователями.
 - *orders*: Общая логика создания и фильтрации заказов.
- *DB (Models/Services)*: Слой доступа к данным, включающий функции инициализации коллекций и формирования AQL-запросов к *ArangoDB*.
- *Services*: Вспомогательная бизнес-логика (такая как, хеширование паролей, автоматическое наполнение БД тестовыми данными).

Используемая архитектура

В проекте реализована архитектура *Client-Server* с четким разделением на:

- *Frontend (React + Tailwind CSS)*: Декларативный подход к UI. Использование React-hooks (*useEffect*, *useState*) для управления состоянием фильтрации и пагинации на стороне клиента.
- *Backend (FastAPI)*: Асинхронная обработка запросов. Используется паттерн *Repository/Service Layer* для работы с базой данных, что позволяет отделить логику хранения данных от логики API.
- *Database (NoSQL)*: Документо-ориентированная модель данных, позволяющая гибко хранить структуру заказов с различными типами мусора без жестких схем.

4.2. Используемые технологии

БД: ArangoDB

Backend: Python 3.11 - alpine

Frontend: React, HTML, Tailwind CSS, JavaScript, Vite.js

4.3. Экраны приложения

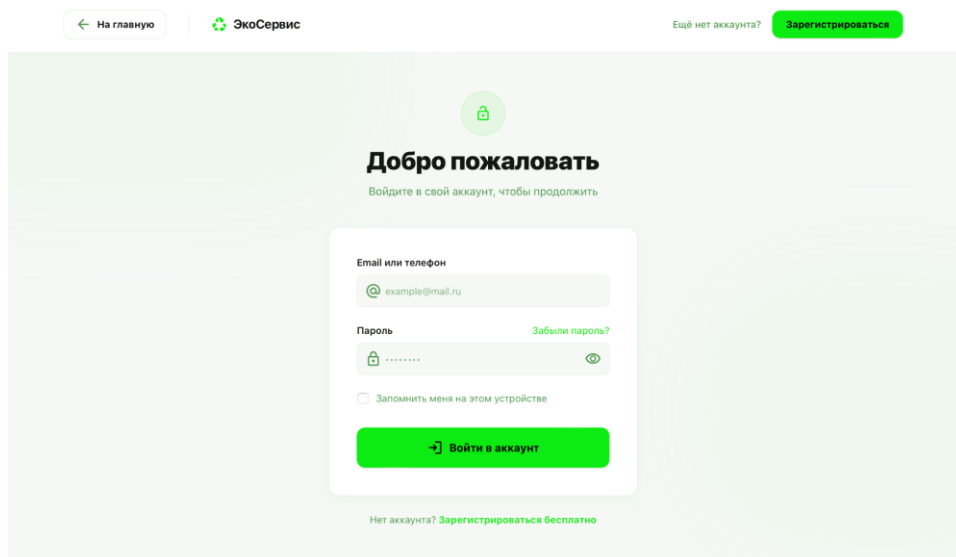


Рисунок 11. Экран входа

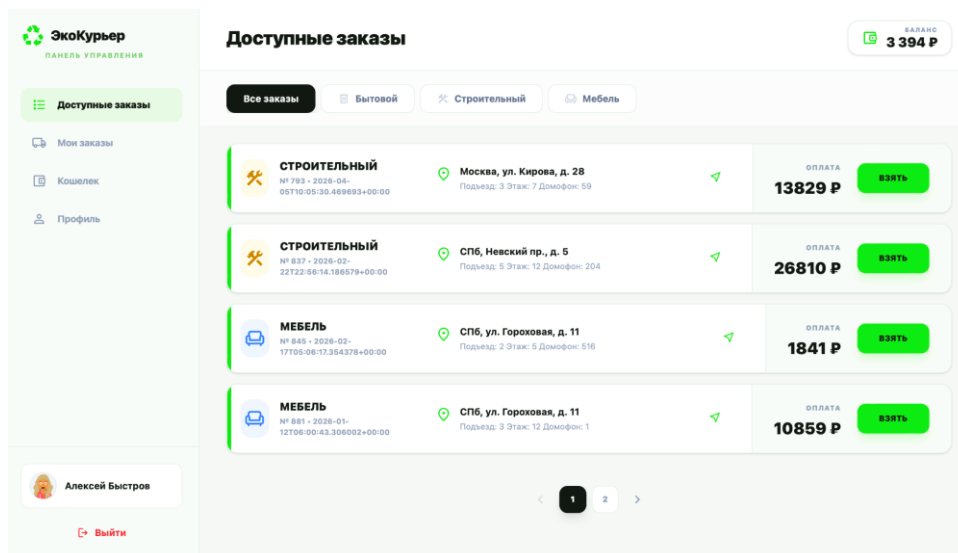


Рисунок 12. Главный экран курьера

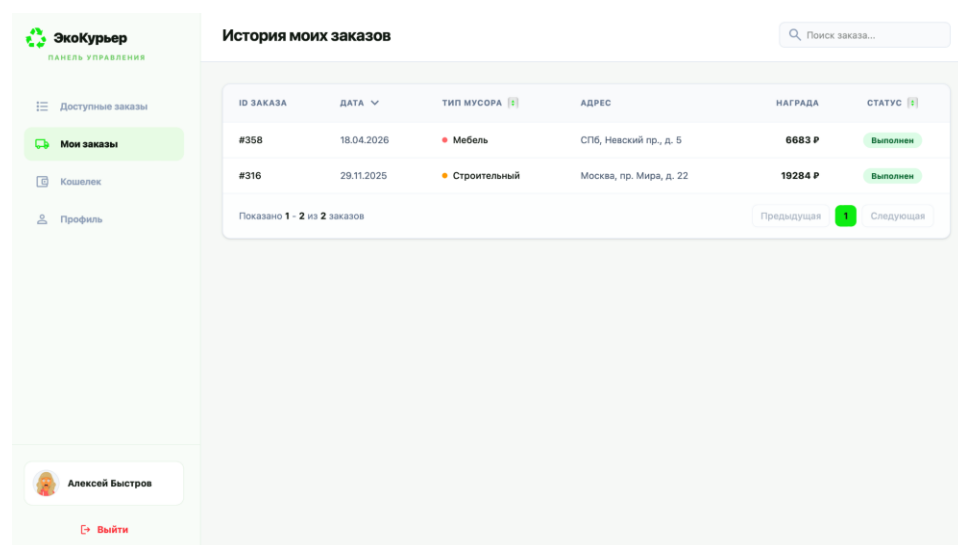


Рисунок 13. Экран с историей заказов курьера

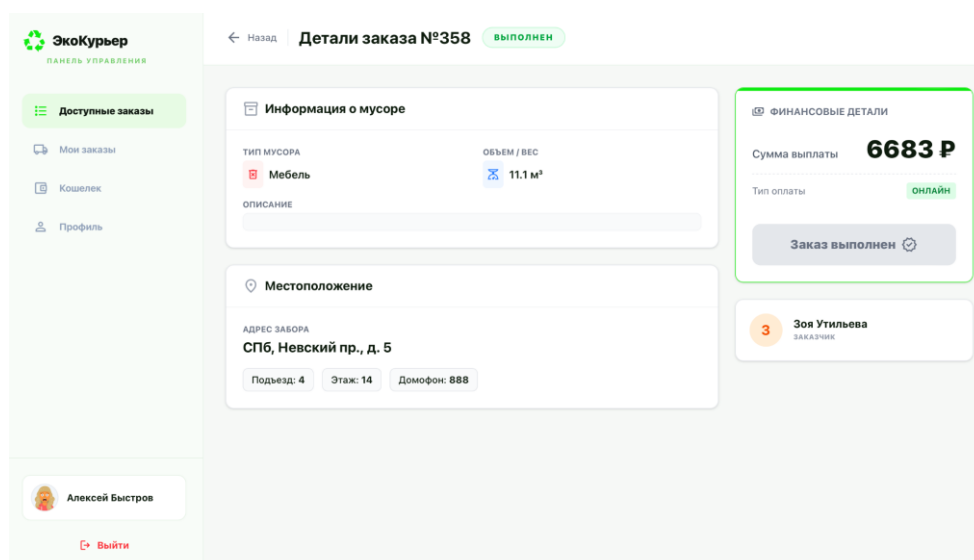


Рисунок 14. Экран с деталями заказа

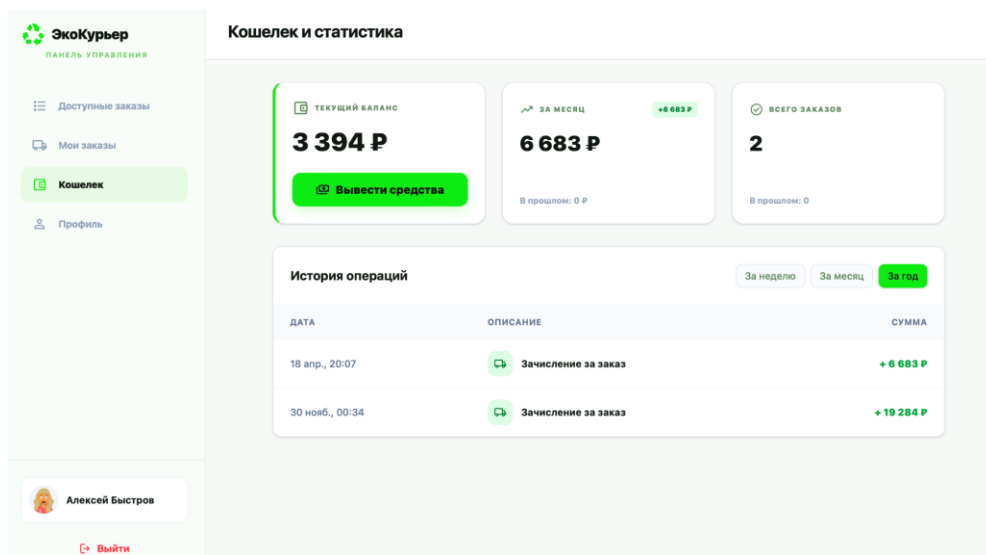


Рисунок 15. Экран транзакций курьера

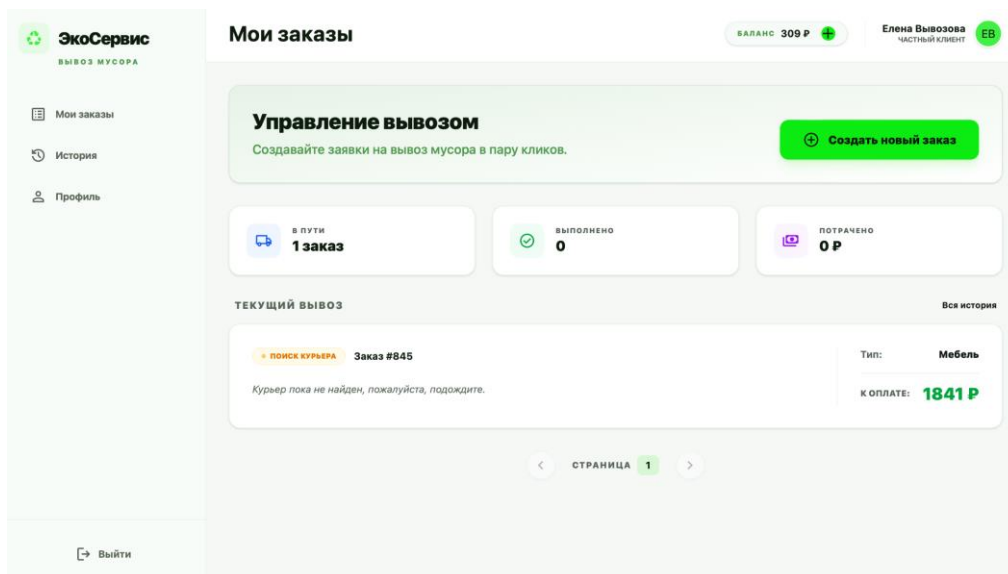


Рисунок 16. Главный экран заказчика

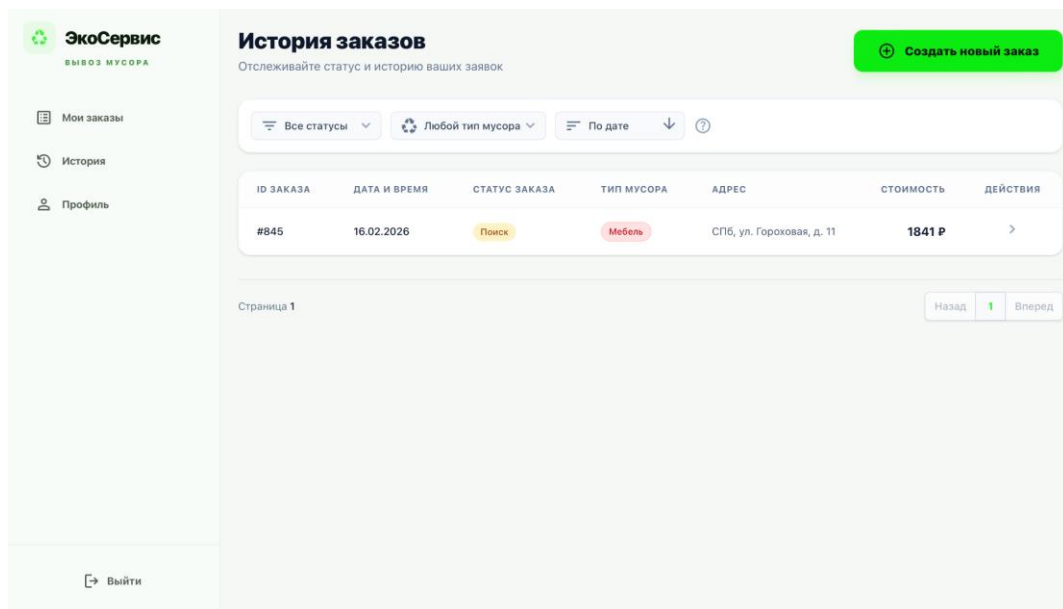


Рисунок 17. Экран с историей заказов у клиента

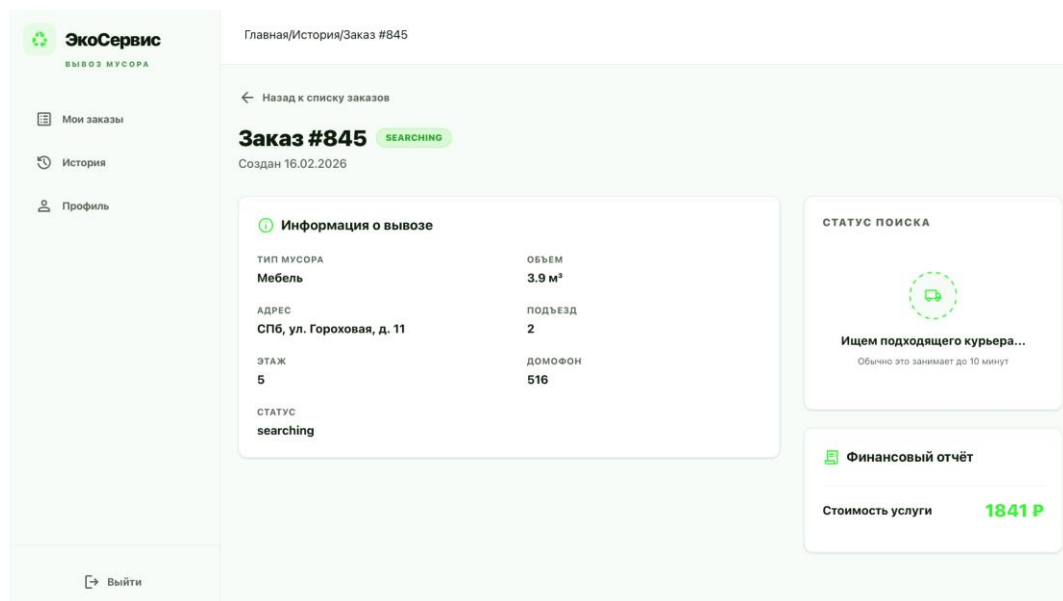


Рисунок 18. Экран с деталями заказа у клиента

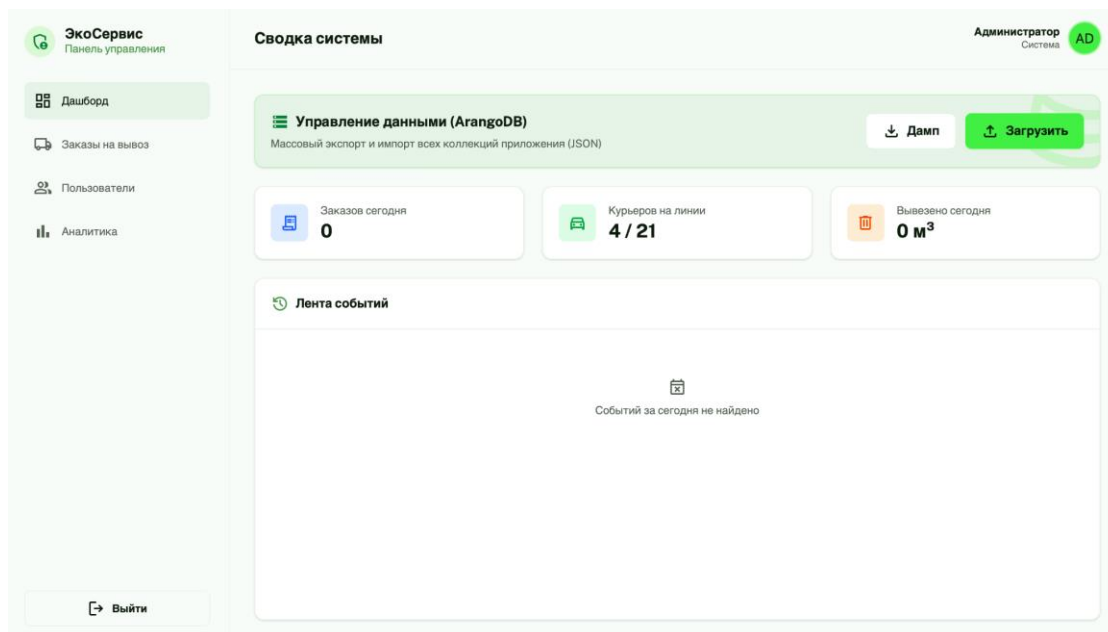


Рисунок 19. Главный экран администратора

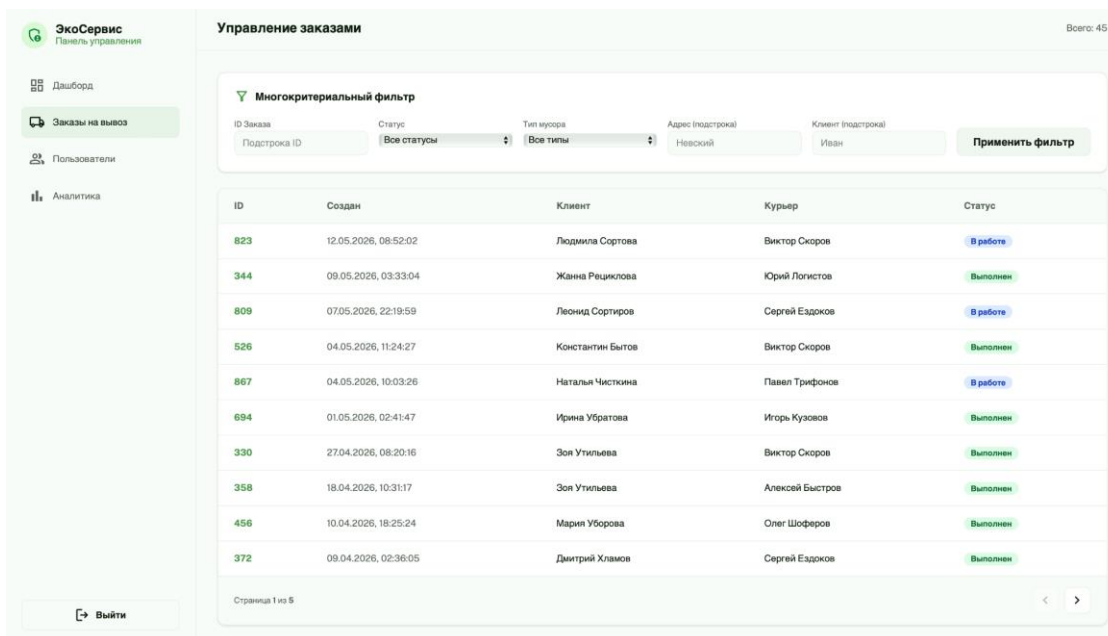


Рисунок 20. Экран с заказами

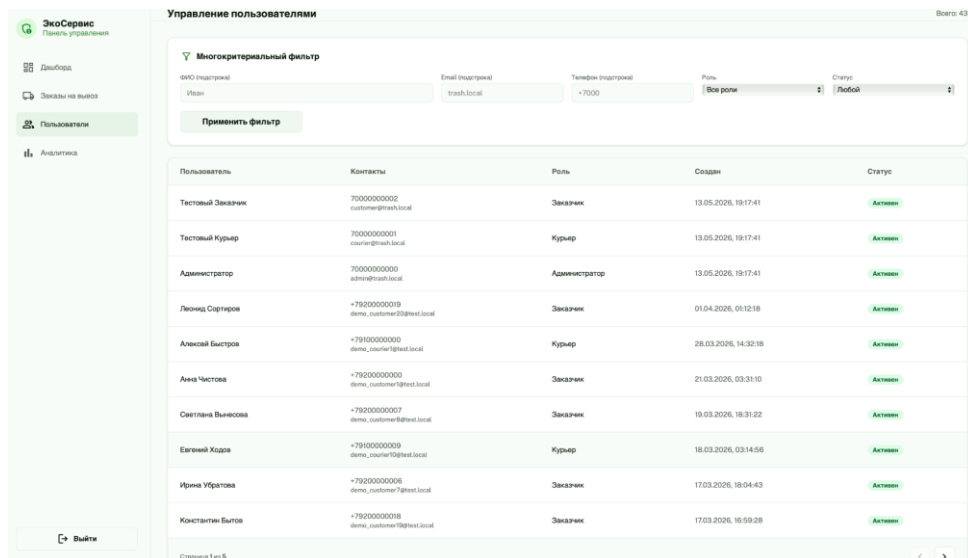


Рисунок 21. Экран с пользователями

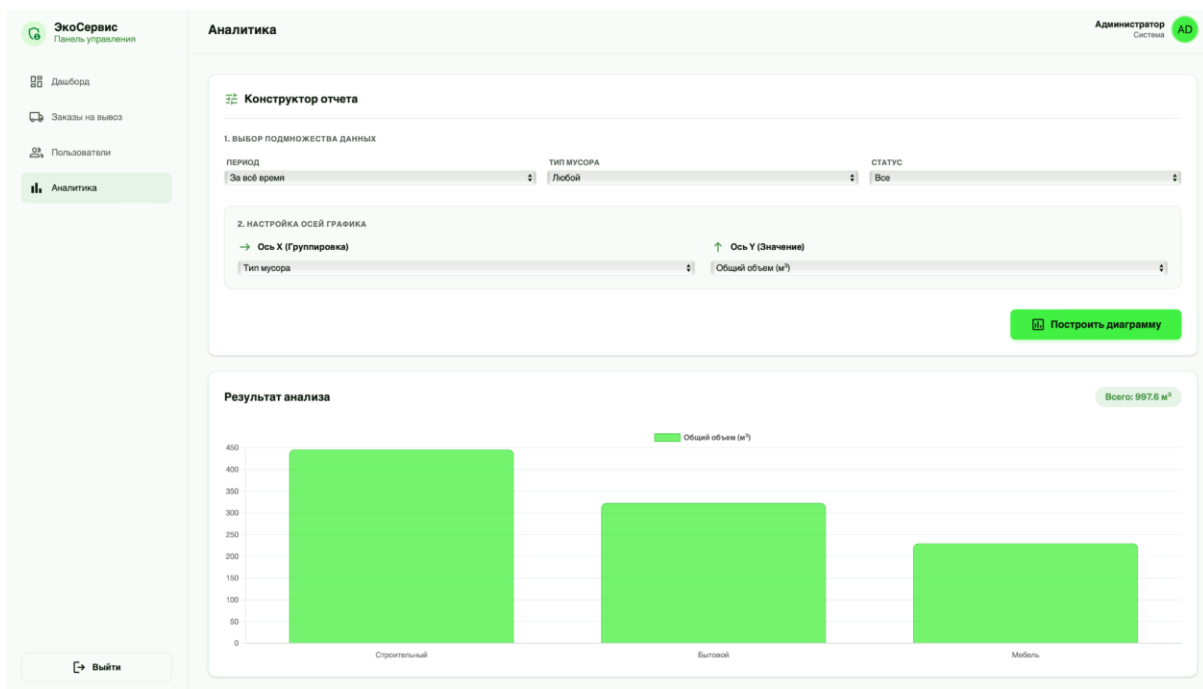


Рисунок 22. Экран с аналитикой

5. ВЫВОД

5.1. Результаты

Разработанный в ходе работы сервис по вывозу мусора, базой данных для которого выступила ArangoDB, предоставляет функционал для трёх

типов пользователей. Заказчики имеют возможность создавать заказы, курьеры могут брать заказы в работу и после выполнения получать оплату, в то время как администратору доступен просмотр различной статистики.

5.2. Недостатки и пути для улучшения полученного решения

В текущей реализации сервиса есть несколько моментов, которые можно оптимизировать для повышения стабильности и удобства:

- *Отсутствие реального картографического сервиса:*
 - *Проблема:* Сейчас адреса заказов хранятся в виде строк.
 - *Улучшение:* Интеграция API (Yandex Maps или OpenStreetMap) для выбора точки на карте и автоматического расчета расстояния.
- *Синхронность уведомлений:*
 - *Проблема:* Пользователю приходится обновлять страницу или ждать срабатывания *useEffect*, чтобы увидеть новый статус заказа.
 - *Улучшение:* Внедрение *WebSockets* для мгновенного обновления статусов в реальном времени без перезагрузки страницы.

5.3. Будущее развитие решения

В будущем можно расширить сервис следующим образом:

- *Система ролей и прав:*
 - Разделение прав администраторов на «супер-админов» и «модераторов» для более безопасного управления пользователями и спорами
- *Интеллектуальная логистика:*

- Добавление алгоритма построения оптимального маршрута для курьера (задача коммивояжера), чтобы он мог забирать несколько заказов за один рейс, исходя из их геопозиции.

6. ПРИЛОЖЕНИЕ

1. Перед запуском убедитесь, что у вас установлен Docker. Для этого выполните следующие две команды в терминале:

```
docker --version  
docker compose version
```

2. Клонировать репозиторий:

```
git clone https://github.com/moevm/nsql1h26-trash
```

3. Перейдите в папку:

```
cd nsql1h26-trash
```

4. Запуск проекта:

```
docker compose build --no-cache  
docker compose up
```

5. Доступ после запуска:

- a. frontend: <http://127.0.0.1:8080>
- b. backend API: <http://127.0.0.1:8000>

ИСПОЛЬЗУЕМАЯ ЛИТЕРАТУРА

1. Arango Documentation // Arango URL: <https://docs.arango.ai>
2. Ссылка на репозиторий: <https://github.com/moevm/nsql1h26-trash>

ПРИЛОЖЕНИЕ А

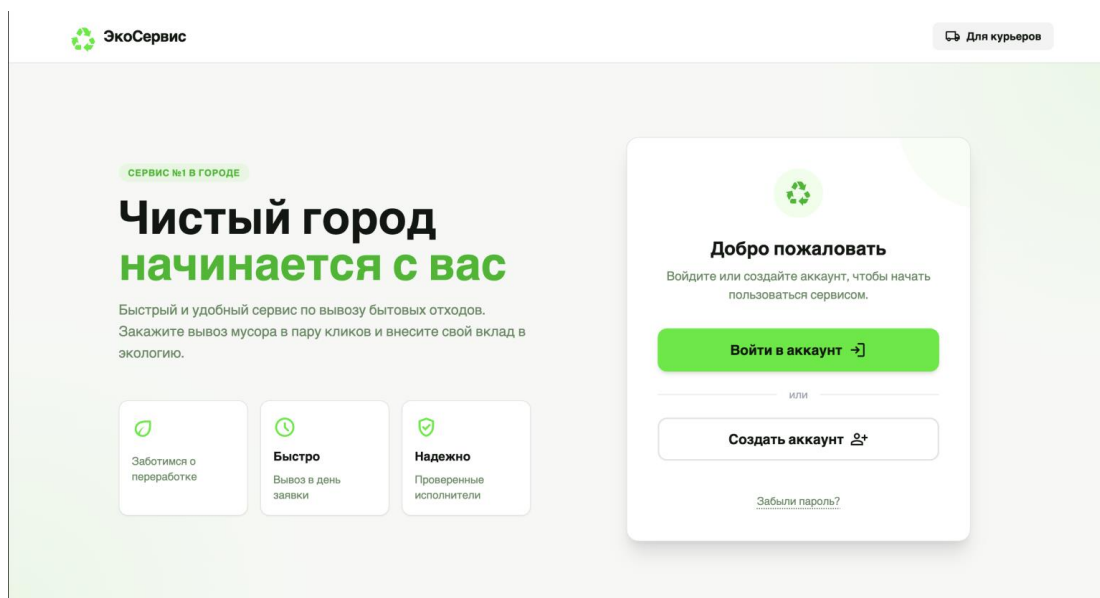


Рисунок А.1. Главная страница

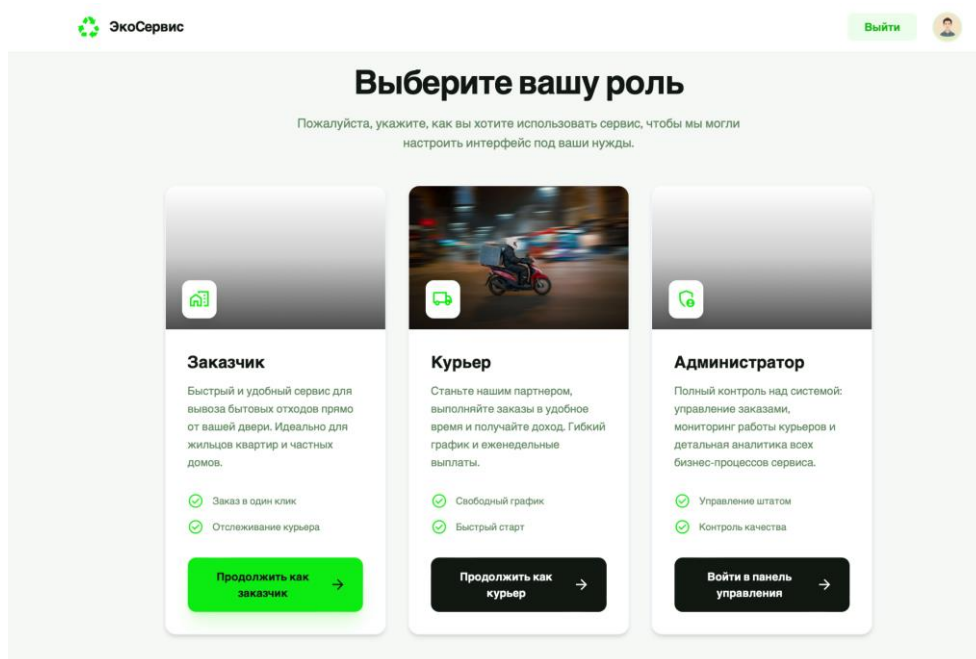


Рисунок А.2. Страница выбора роли

← Назад |  ЭкоСервис

Регистрация

Создайте аккаунт для управления вывозом мусора

ФИО

 Иванов Иван Иванович

ТЕЛЕФОН

 +7 (000) 000-00-00

ЭЛЕКТРОННАЯ ПОЧТА

 example@mail.ru

АДРЕС ПРОЖИВАНИЯ

 г. Санкт-Петербург, ул. Мира, д. 10

ПАРОЛЬ





ПОДТВЕРДИТЕ ПАРОЛЬ



☐ Я согласен с [Условиями использования](#) и [Политикой конфиденциальности](#)

Зарегистрироваться

Уже есть аккаунт? [Войти](#)

Рисунок А.3. регистрация заказчика

← Назад |  ЭкоСервис

Стать курьером

Заполните форму, чтобы начать зарабатывать.

ФИО

 Иванов Иван Иванович

ТЕЛЕФОН

 +7 (000) 000-00-00

ГОРОД

 Санкт-Петербург

ЭЛЕКТРОННАЯ ПОЧТА

 example@mail.ru

ПАРОЛЬ





ПОДТВЕРЖДЕНИЕ



ТРАНСПОРТ

 Выберите тип транспорта

ФОТО ПАСПОРТА



Нажмите для загрузки

Зарегистрироваться

Нажимая на кнопку, вы соглашаетесь с [Условиями использования](#) и [Политикой конфиденциальности](#)

Рисунок А.4. Регистрация курьера

← Назад  ЭкоСервис

Регистрация

Создайте аккаунт для управления вывозом мусора

ФИО

ТЕЛЕФОН

ЭЛЕКТРОННАЯ ПОЧТА

ПАРОЛЬ

ПОДТВЕРДИТЕ ПАРОЛЬ

☐ Я согласен с [Условиями использования](#) и [Политикой конфиденциальности](#)

Зарегистрироваться

[Уже есть аккаунт? Войти](#)

Рисунок А.5. Регистрация администратора

← На главную  ЭкоСервис [Ещё нет аккаунта? Зарегистрироваться](#)

Добро пожаловать

Войдите в свой аккаунт, чтобы продолжить

Email или телефон

Пароль

[Забыли пароль?](#)

☐ Запомнить меня на этом устройстве

Войти в аккаунт

[Нет аккаунта? Зарегистрироваться бесплатно](#)

Рисунок А.6. Вход

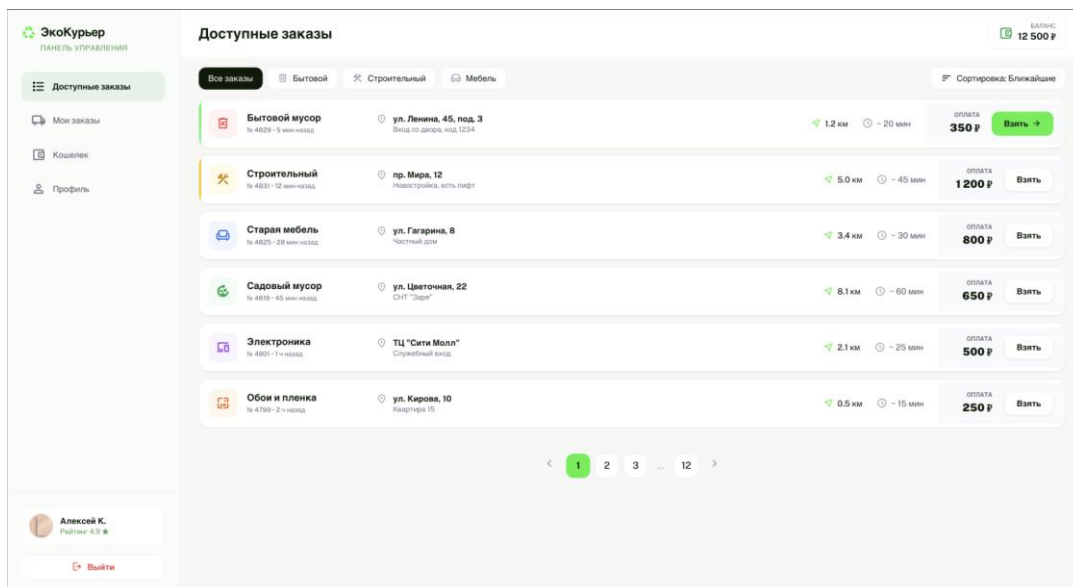


Рисунок А.7. Главная страница курьера

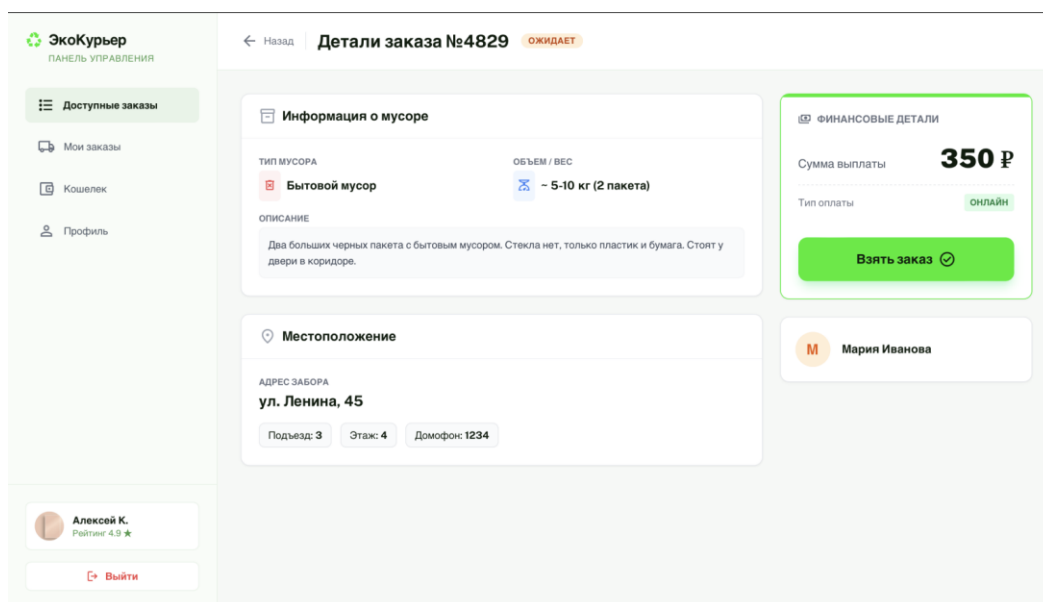


Рисунок А.8. Взятие заказа

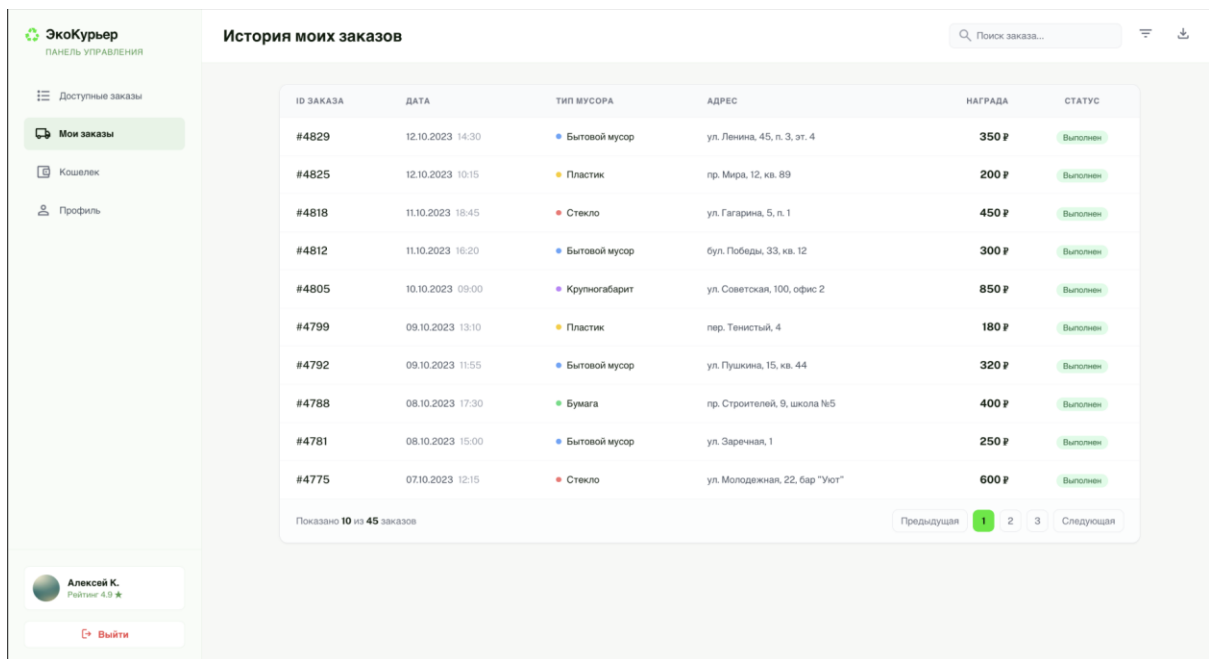


Рисунок А.9. История заказов

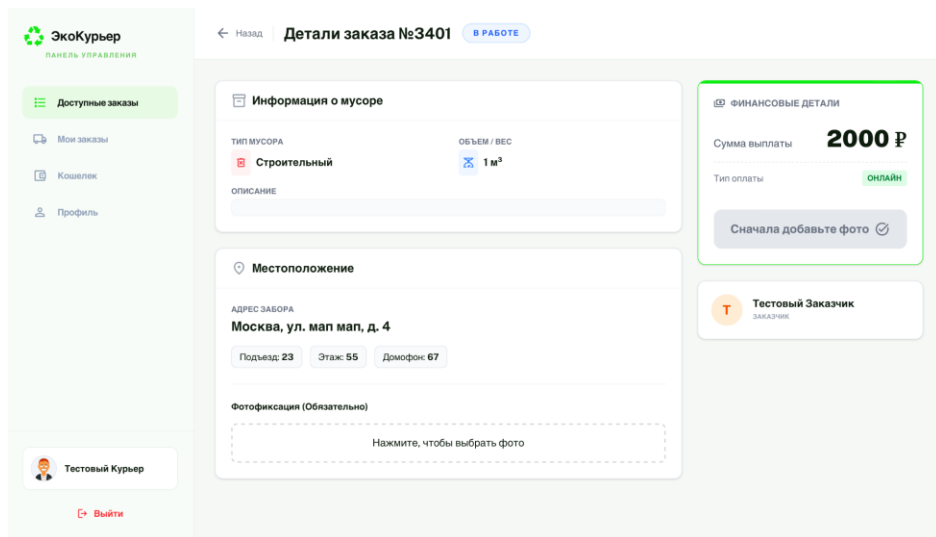


Рисунок А.10. Подтверждение выполнения заказа

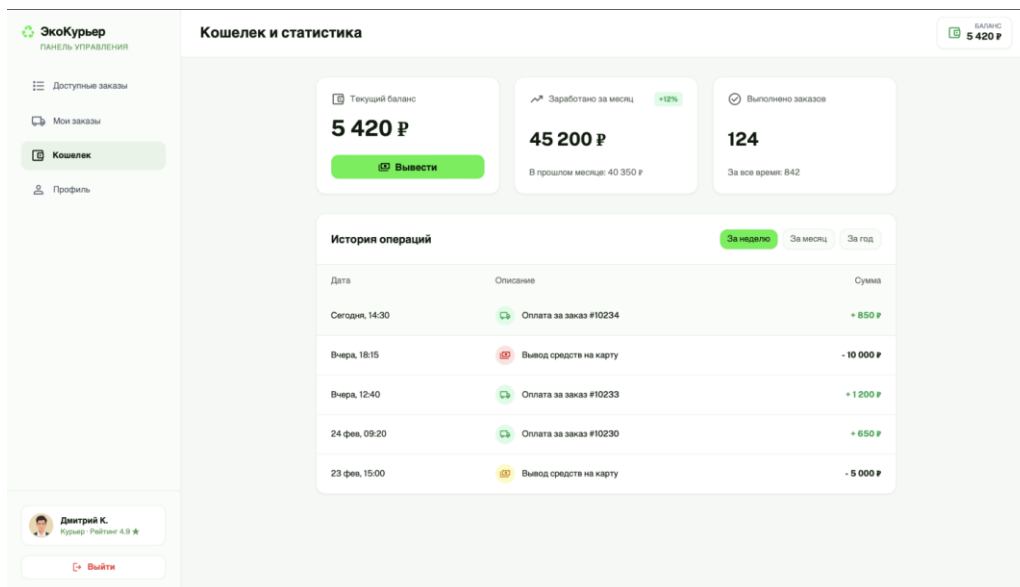


Рисунок А.11. Страница с транзакциями и статистикой

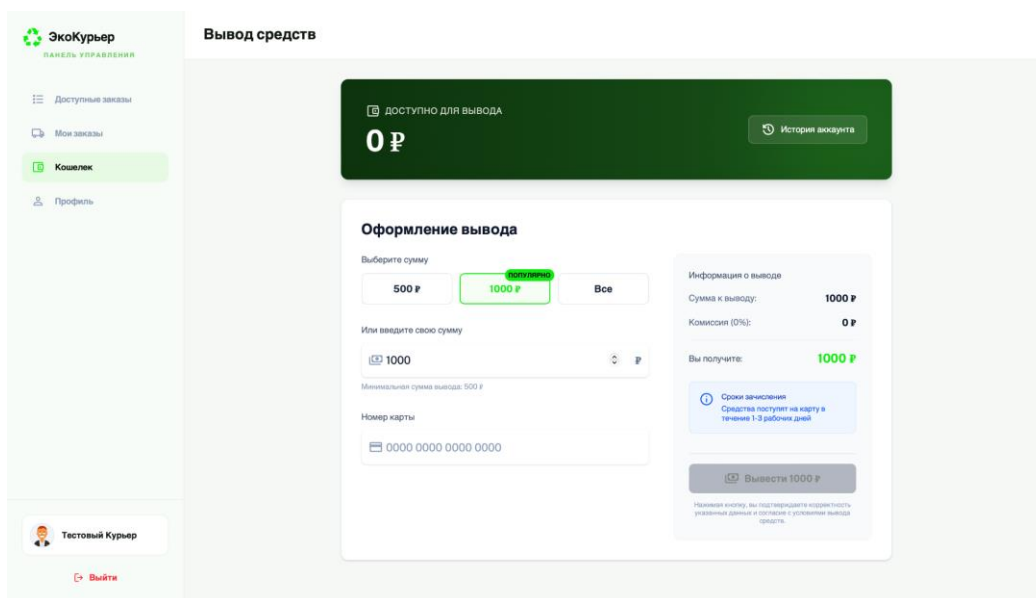


Рисунок А.12. Вывод денежных средств

ЭкоКурьер

ПАНЕЛЬ УПРАВЛЕНИЯ

Доступные заказы

Мои заказы

Кошелек

Профиль

Дмитрий К.

Курьер · Рейтинг 4.9 ★

Выйти

Личный профиль

Баланс 5 420 ₽

Изменить фото

Имя

Дмитрий

Фамилия

Ковалев

Номер телефона

+7 (905) 123-45-67

Электронная почта

dmitry.k@example.com

Транспортное средство

Пешком

На авто

На газели

Сменить пароль

Сохранить изменения

Рисунок А.13. Профиль курьера

ЭкоСервис

Профиль

Смена пароля

Для защиты вашего аккаунта используйте сложный пароль, состоящий из букв и цифр.

Старый пароль

Введите старый пароль

Новый пароль

Введите новый пароль

Подтвердите новый пароль

Повторите новый пароль

Минимум 8 символов, заглавные буквы и цифры

Обновить пароль

Отмена

Рисунок А.14. Смена пароля

65

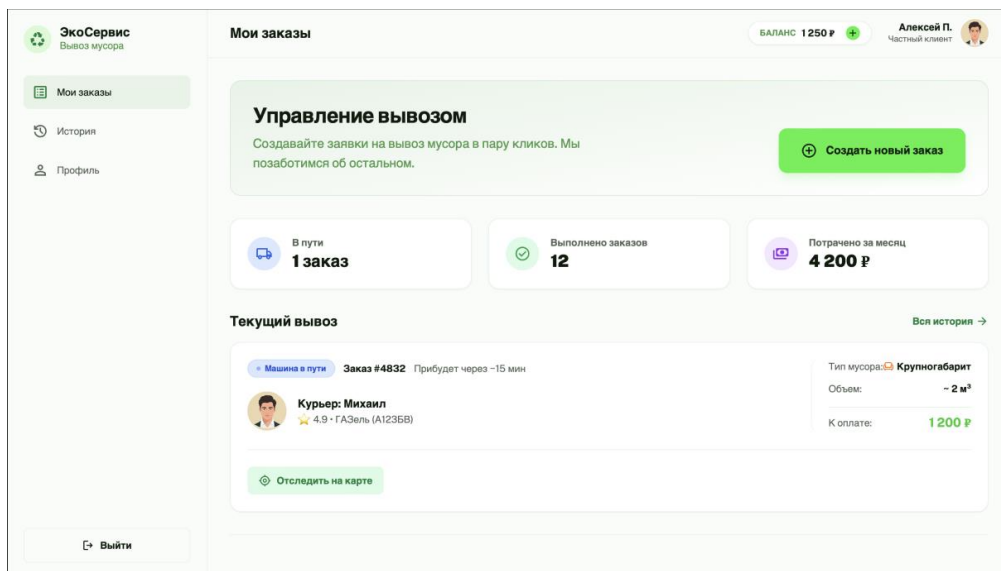


Рисунок А.15. Основная страница заказчик

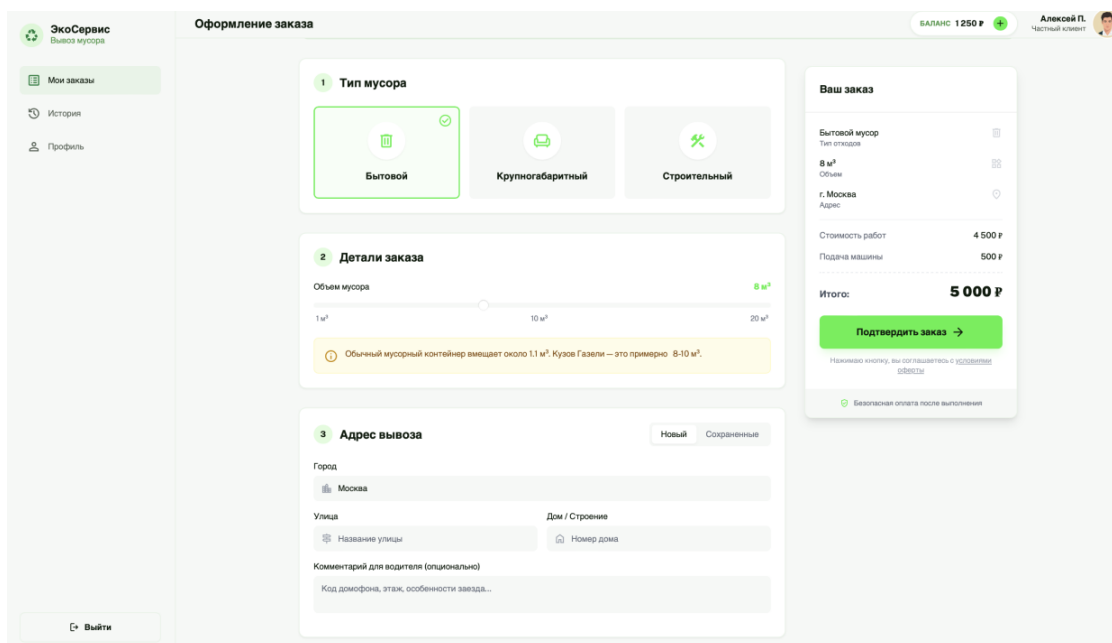


Рисунок А.16. Создание заказа

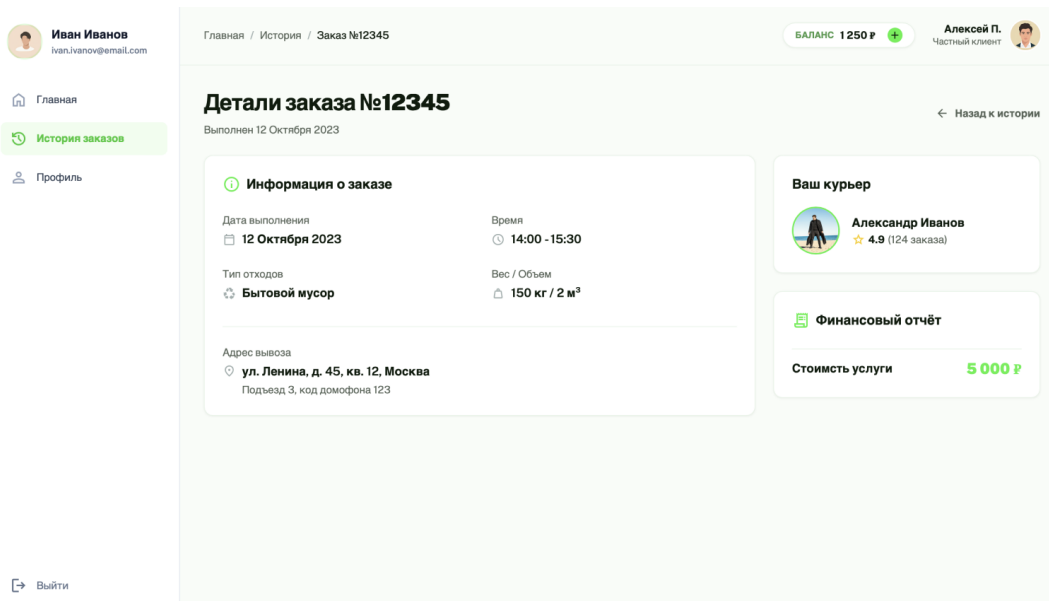


Рисунок А.17. Просмотр деталей заказа

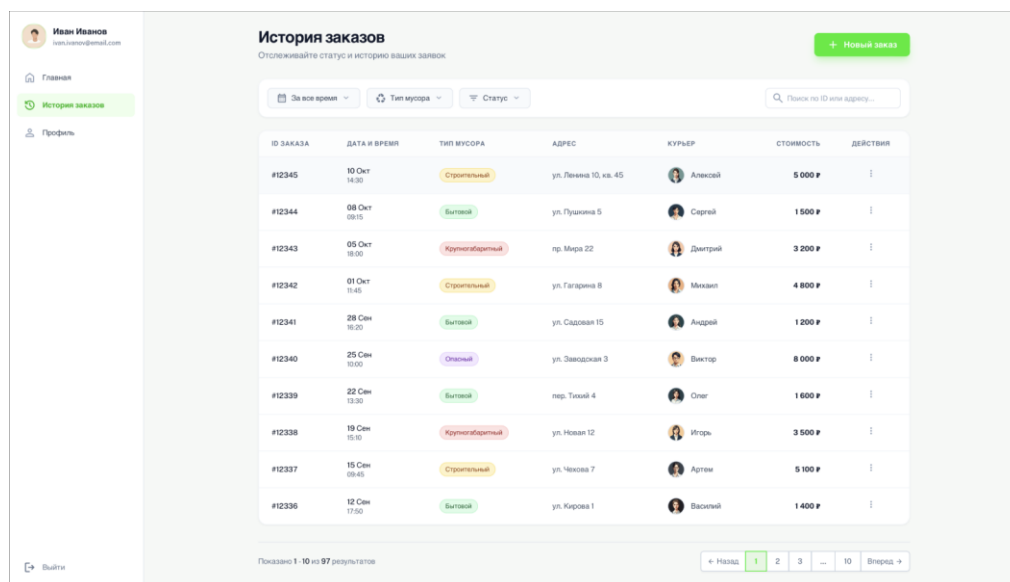


Рисунок А.18. Все заказы

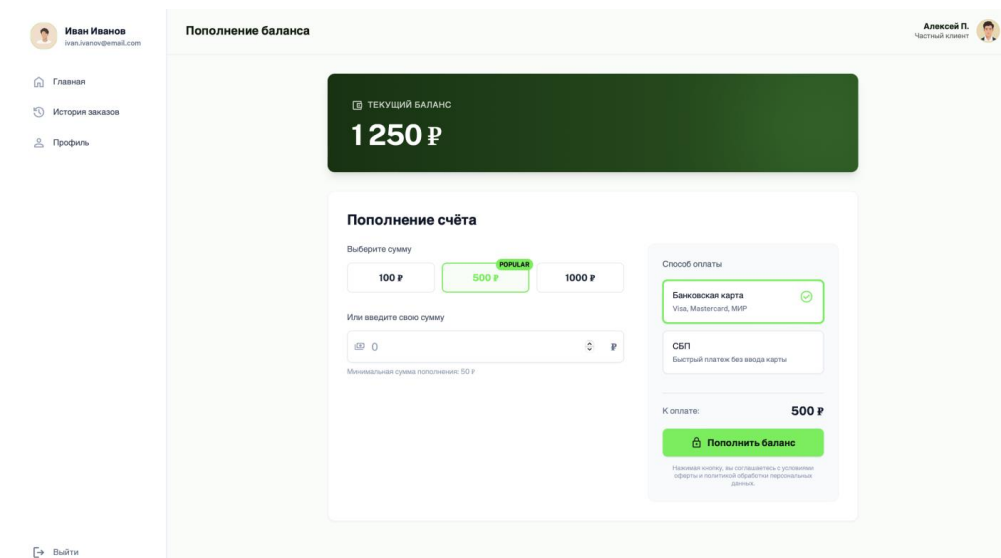


Рисунок А.19. Пополнение счёта

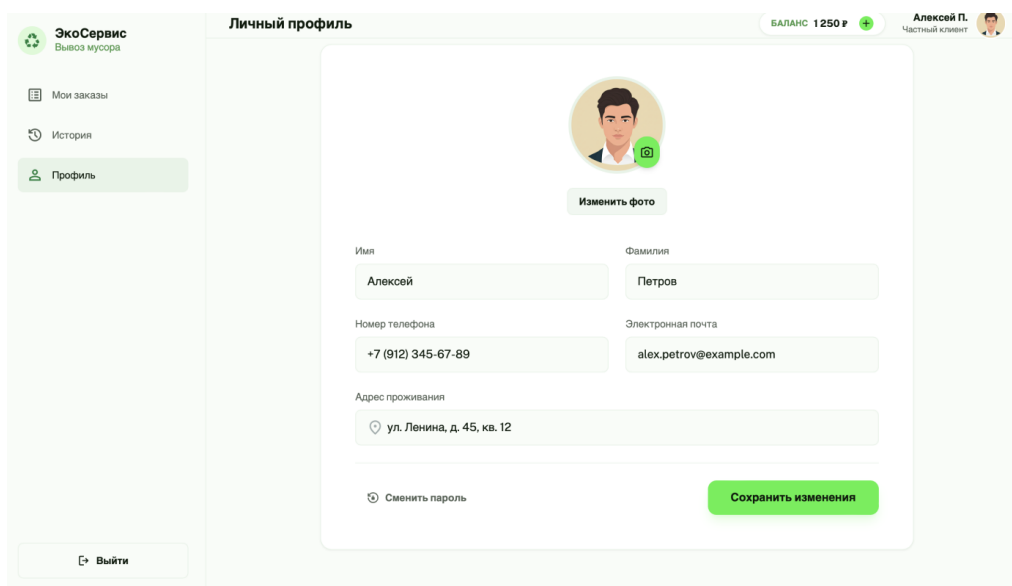


Рисунок А.20. Профиль заказчика

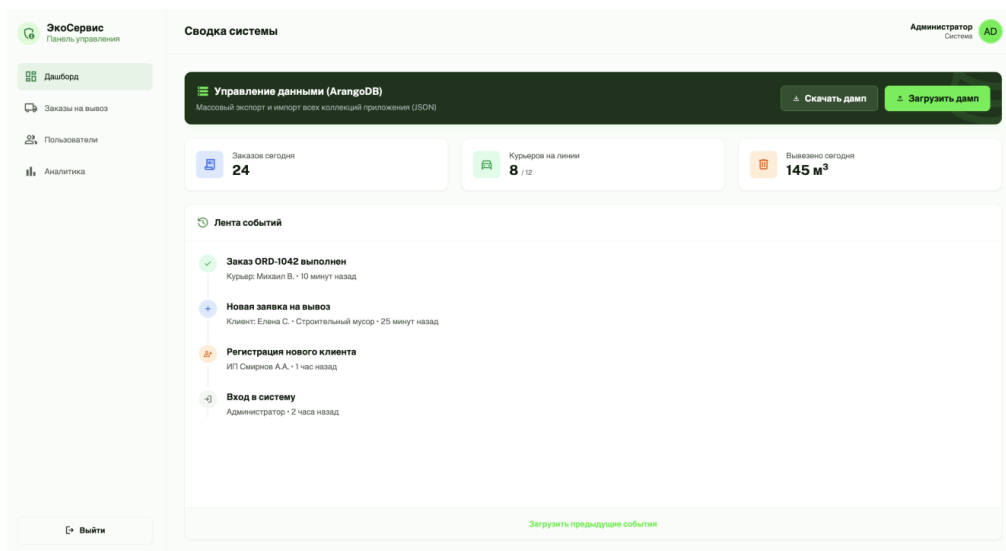


Рисунок А.21. Основная страница администратора

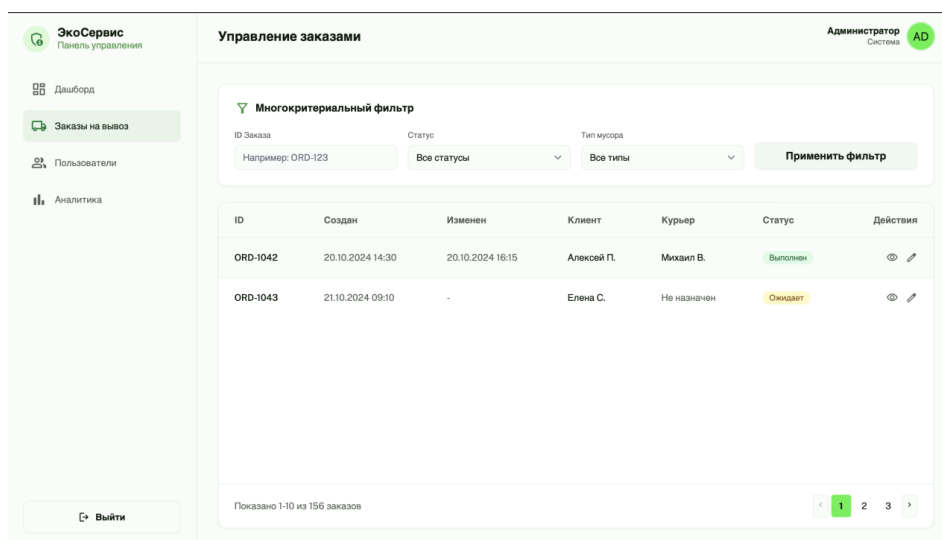


Рисунок А.22. Управление заказами

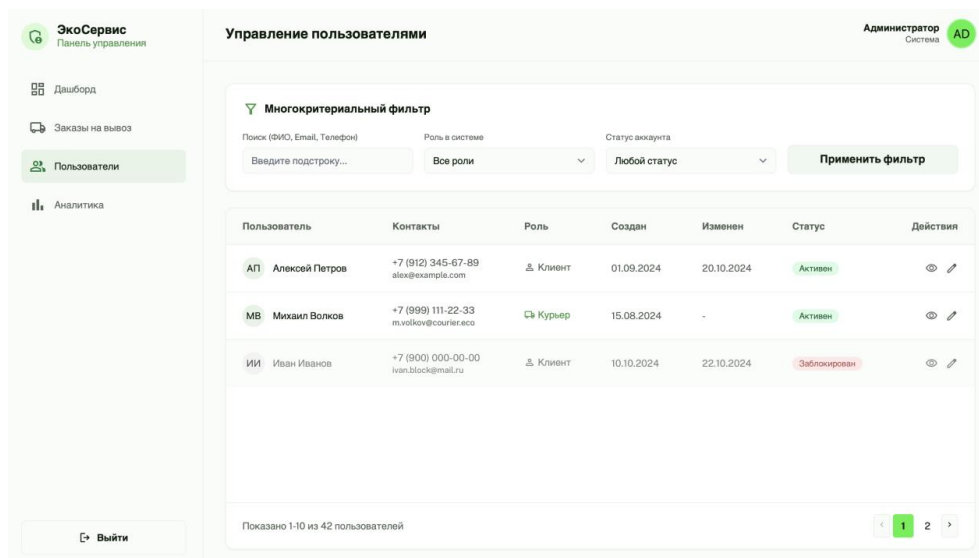


Рисунок А.23. Управление пользователями

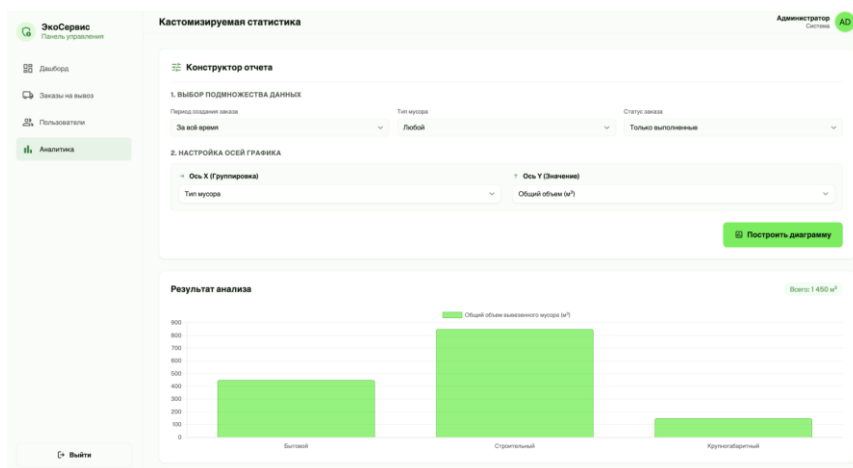


Рисунок А.24. Статистика

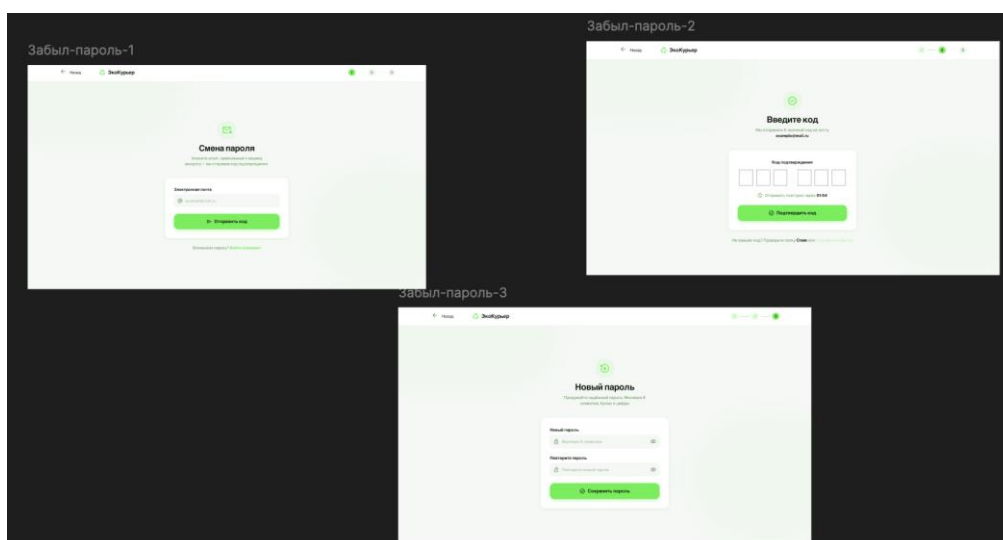


Рисунок А.25. Восстановление пароля